Join at slido.com
#1846379

Lecture 12

Optimization and Gradient Descent

The core algorithm of modern machine learning

EECS 189/289, Fall 2025 @ UC Berkeley
Joseph E. Gonzalez and Narges Norouzi

Key Factors in Generative AI Revolution

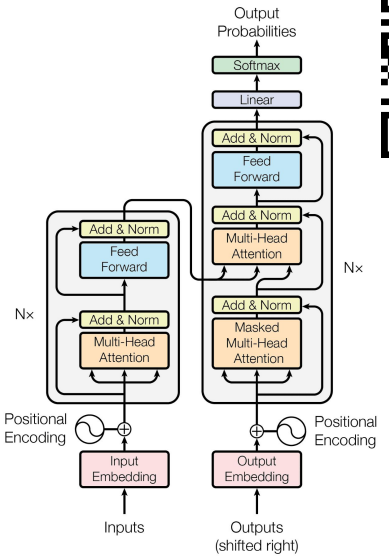
1846379

Data

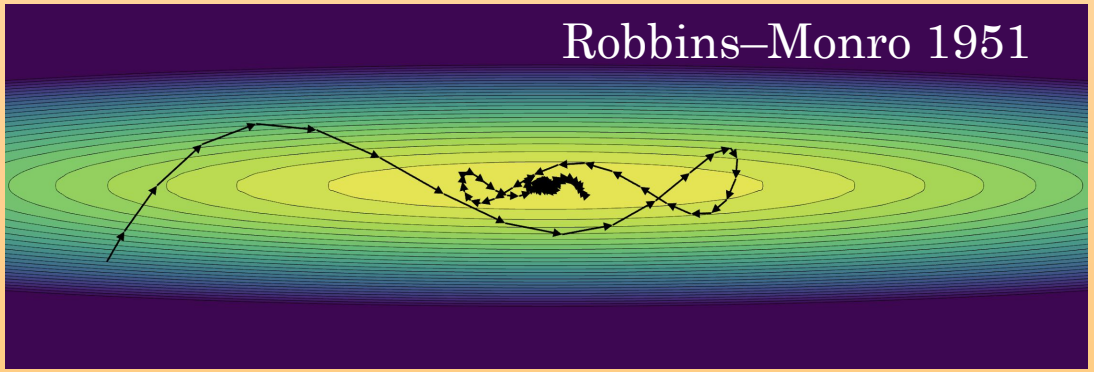
$$\ln p\left(\mathrm{Word}_n \mid \{\mathrm{Word}_i\}_{i=1}^{n-1}\right)$$

Loss Functions

Compute



Neural Architectures



Stochastic Gradient Descent

PyTorch

TensorFlow

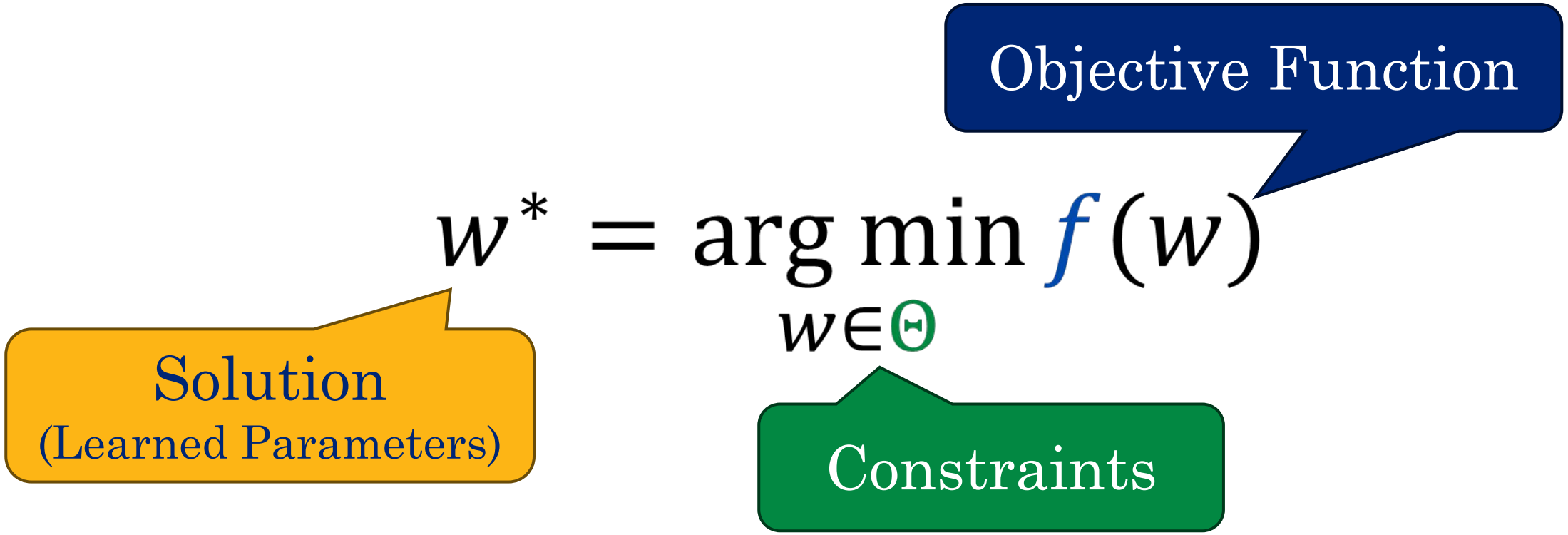
Automatic Diff.

The General Optimization Problem

The General Optimization Problem

1846379

Most problems in ML can be reduced to the following general **optimization problem**



Optimization Constraints

1846379

The **constraints** define the valid set of parameters.

$$w^* = \arg\min_{w \in \Theta} f(w)$$

Constraints

For the rest of this lecture, we will focus on the **unconstrained setting.**

$$\Theta = \mathbb{R}^d$$

The most common case in ML is **unconstrained optimization** in which $\Theta = \mathbb{R}^d$.

In some cases, there may be integer (e.g., $\Theta = \mathbb{Z}^d$) or probability constraints:

$$\Theta = \left\{ w \mid w \in \mathbb{R}^d, 0 \leq w \leq 1, \sum w_i = 1 \right\}$$

The Objective Function

1846379

The **shape** of the objective function plays a big role in the complexity of solving the optimization problem.

Objective Function

$$w^* = \arg\min_{w \in \Theta} f(w)$$

The **Objective Function** in ML is often referred to as the **loss** or **error** function and depends on the **data.**

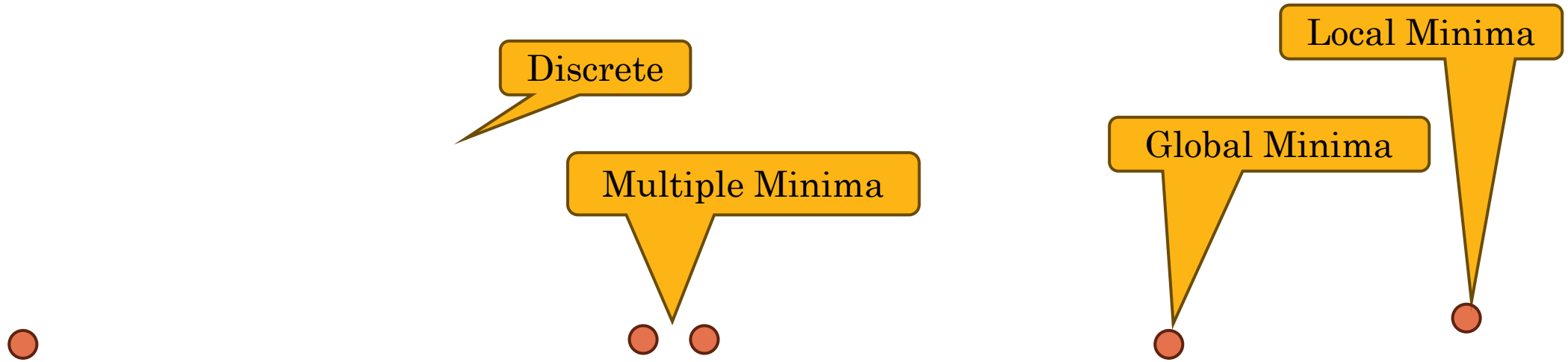
$$f(w) = E[w; \mathfrak{D}]$$

Examples Optimization Problems

$$\arg\min_{w \in \mathbb{R}} w^2 - 3w + 4$$

$$\arg\min_{w \in \mathbb{Z}} w^2 - 3w + 4$$

$$\arg\min_{w \in \mathbb{R}} w^4 - 5w^2 + w + 4$$



How do we solve these optimization problems?

What properties of the objective function make it "easier" to solve?

Plots in demo notebook.

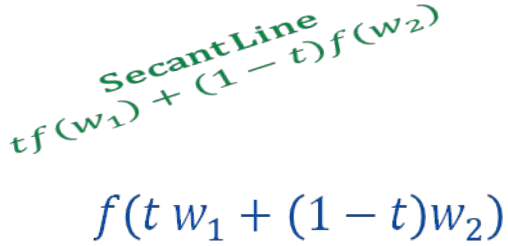
Convex Functions

1846379

Definition: A function is **convex** on the domain Θ if for any pair of points $\forall\, w_1, w_2 \in \Theta$ and for all $0 \leq t \leq 1$:

$$f(t\, w_1 + (1-t)w_2) \leq tf(w_1) + (1-t)f(w_2)$$

Convex:



Non-Convex:

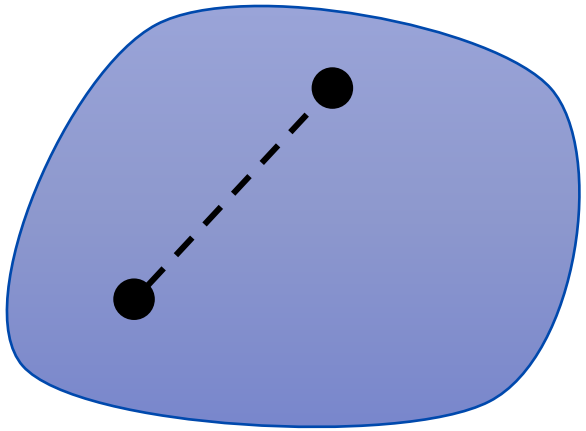
In general, there are **efficient algorithms** to reliably minimize convex functions.

Convex (Constraint) Sets

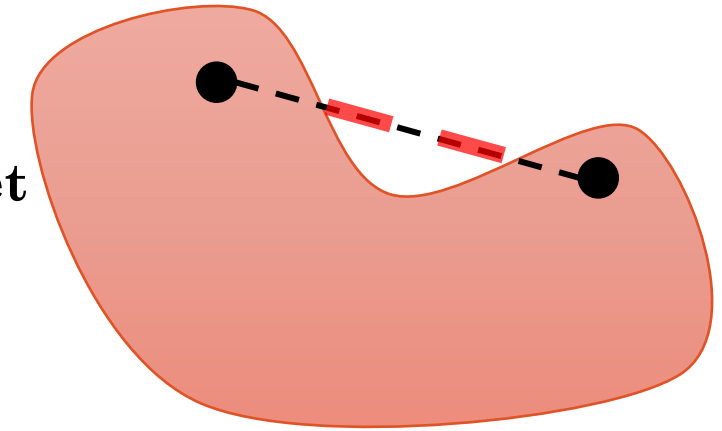
Definition: A set S is convex if it contains the **line segment** between **any two points** in the set:

$$\forall w_1, w_2 \in S, ;\ 0 \leq t \leq 1 :\quad t\, w_1 + (1-t) w_2 \in S$$

Convex Set



Non-convex Set



In general, there are **efficient algorithms** to reliably minimize **convex functions** over **convex sets**.

Optimization Problems

1846379

Optimization of a **convex objective function** over a **convex constraint set**.

- *ML problems that we can solve efficiently*
- Primary focus of EECS-127 and books

Convex Function

Convex Constraints

Many classic ML problems are **convex**.

- **Least Squares Regression** and **Logistic Regression**

Most of **modern ML (deep learning) is not convex**.

- We use convex optimization **techniques** for non-convex problems
- In this class, introduce basic **optimization concepts**

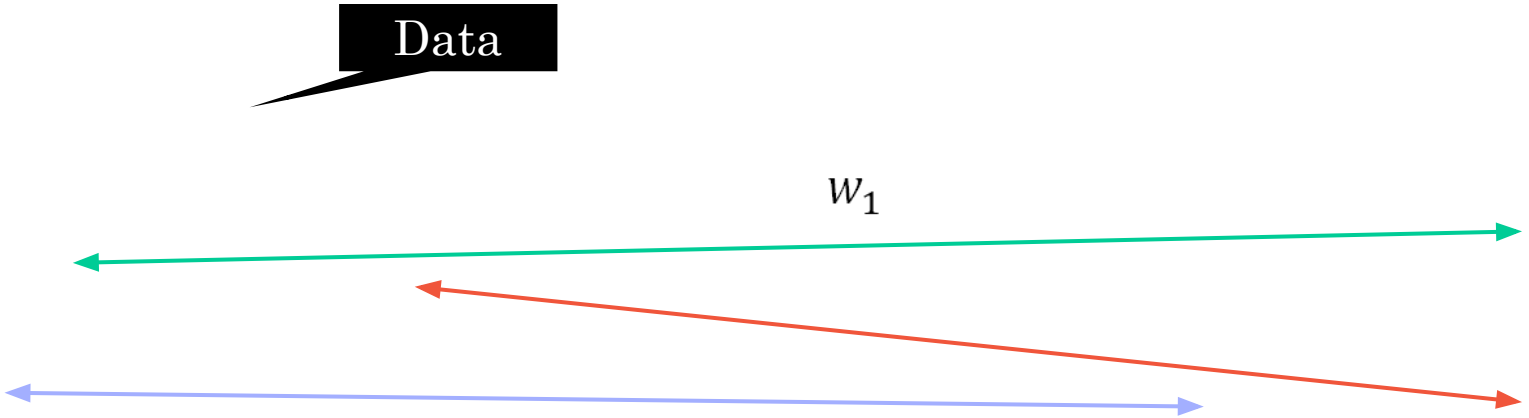
Minimizing the Error & Optimizing in Weight-Space

Points on the Error Surface

- Each **parameterization of a model** corresponds to a point on the error surface $E(w; D)$.

w_1

w_0

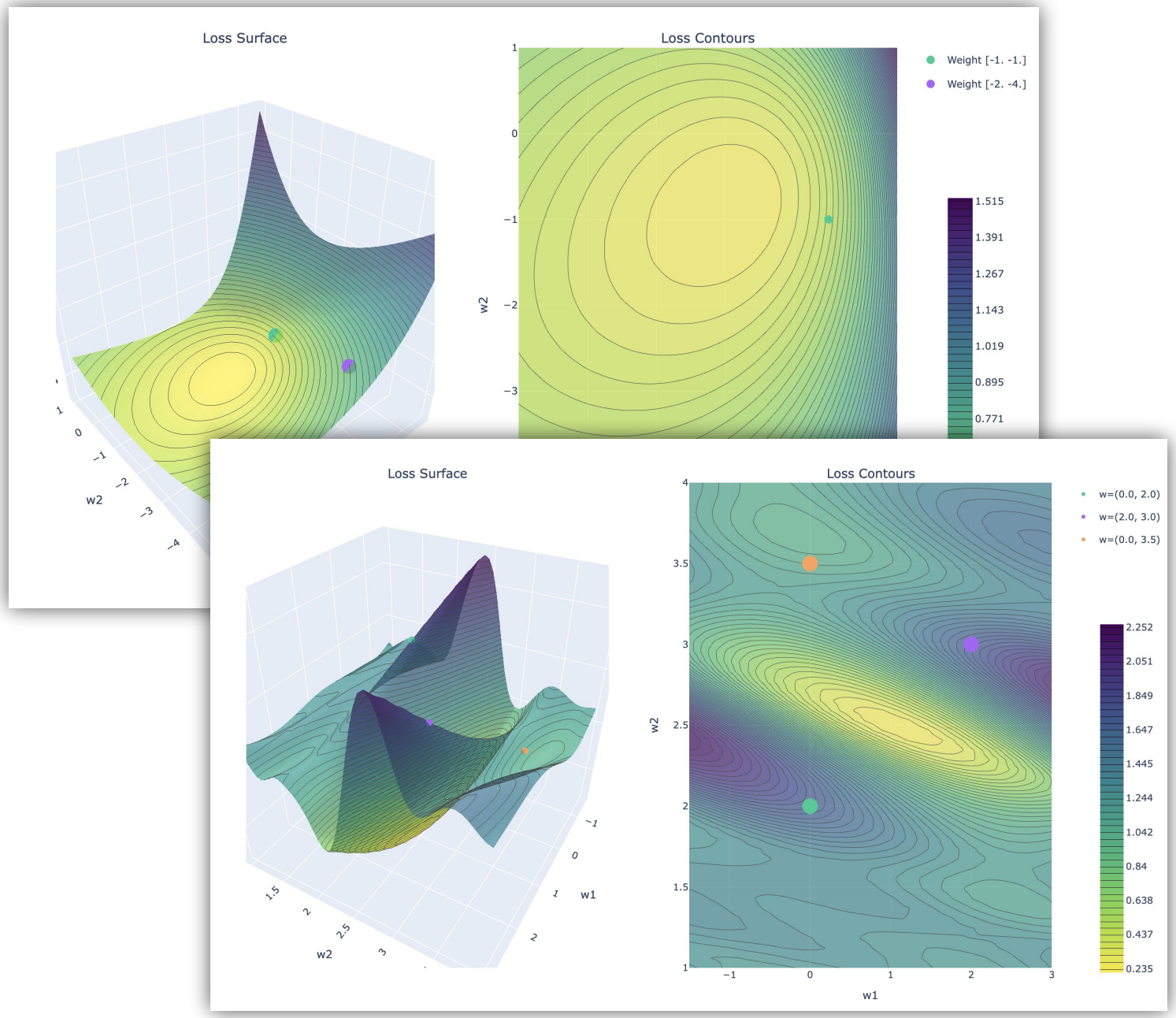


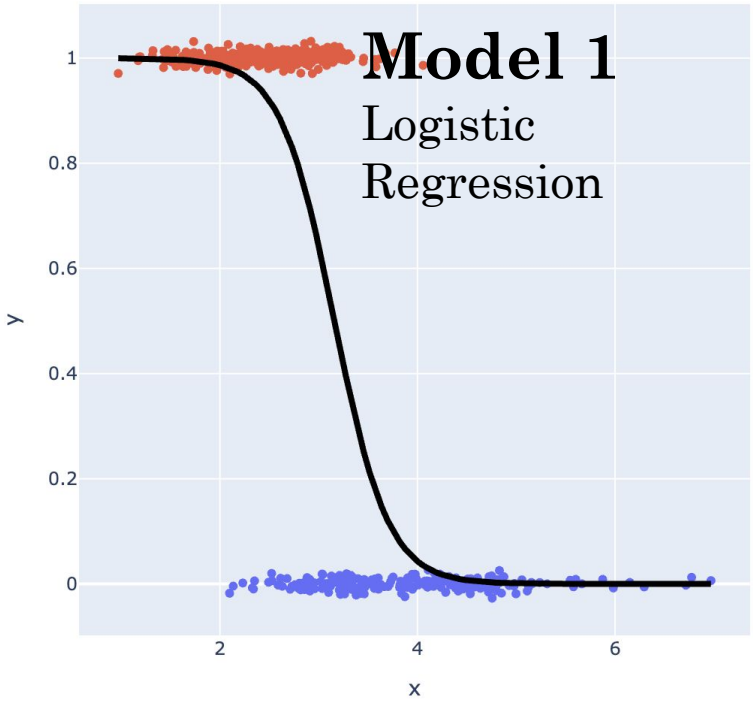
Goal: find parameterization with lowest error: $w^* = \arg\min_{w \in \mathbb{R}^d} E(w; D)$

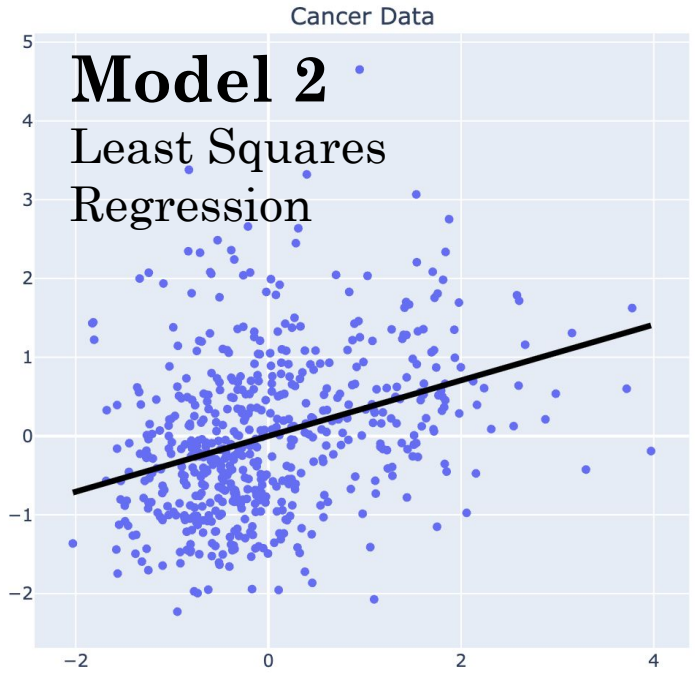
Plots in demo notebook.

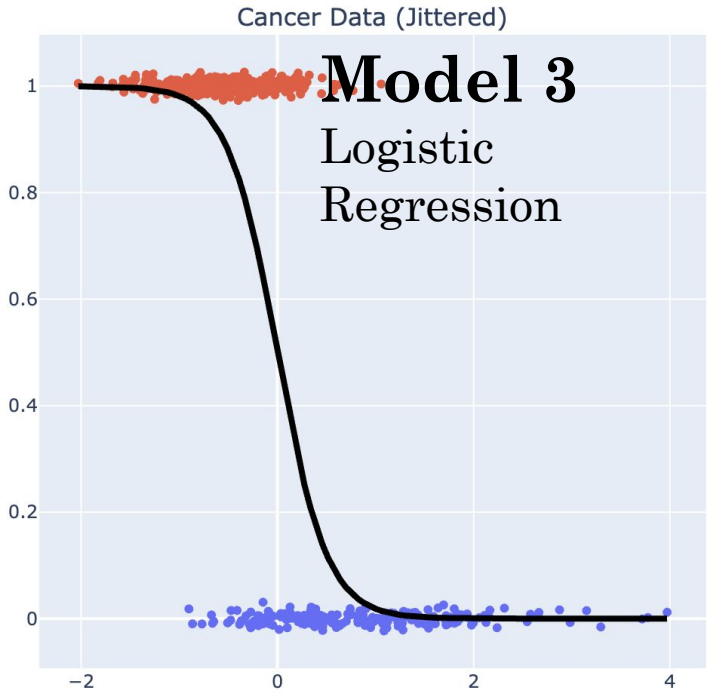
Demo

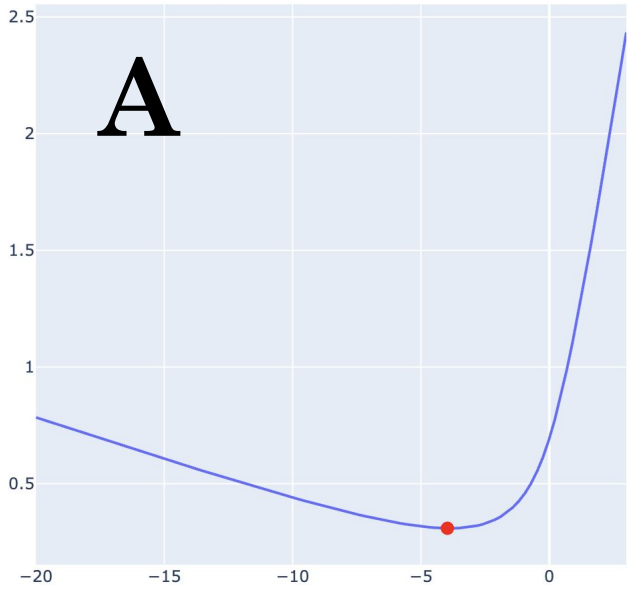
Understanding the Error Surface

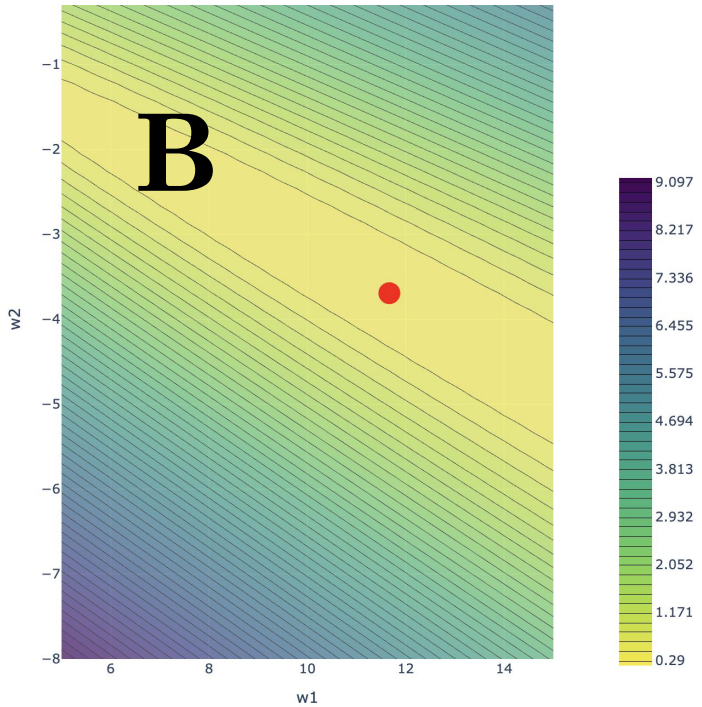


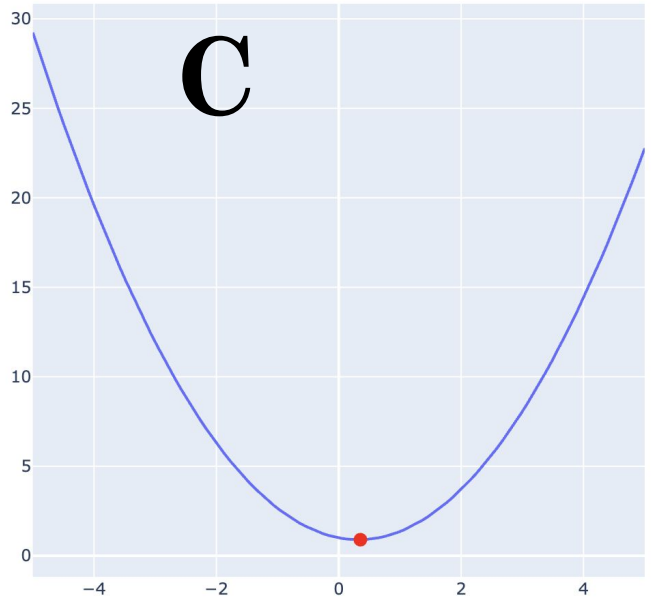

Model 1
Logistic
Regression
y
x


Cancer Data
Model 2
Least Squares
Regression


Cancer Data (Jittered)
Model 3
Logistic
Regression


A


B
w1
w2
9.097
8.217
7.336
6.455
5.575
4.694
3.813
2.932
2.052
1.171
0.29


C

Match the model with the loss.

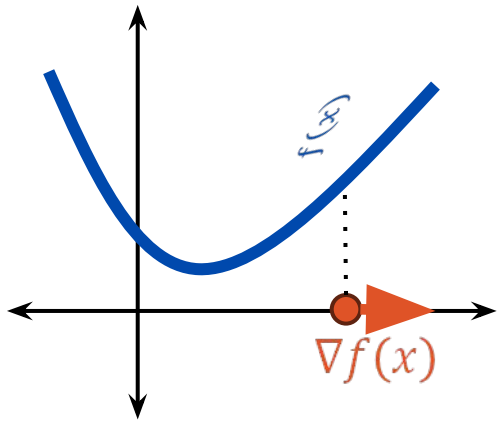
Calculus and Optimization

The Gradient

1846379

The gradient of a **scalar-valued function** $f: \mathbb{R}^d \to \mathbb{R}$ is the **column vector-valued function** $\nabla f: \mathbb{R}^d \to \mathbb{R}^d$ of partial derivatives of f

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}$$



Important Properties

- The gradient $\nabla f(x)$ points in the **direction of steepest ascent**.
- The gradient $\nabla f(x) = 0$ at local minima, maxima, and saddle points.
  - We already used this property to solve for the MLE (…many times…)

Gradient Exercise

Gradient

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_d} \end{bmatrix}$$

Consider the function:

$$E(w) = (w_0 - 1)^2 + (w_1 - 2)^2 + 1$$

Compute the partial derivatives:

$$\frac{\partial}{\partial w_0} E(w) = 2(w_0 - 1)$$

$$\frac{\partial}{\partial w_1} E(w) = 2(w_1 - 2)$$

Then the gradient is:

$$\nabla E(w) = \begin{bmatrix} 2(w_0 - 1) \\ 2(w_1 - 2) \end{bmatrix}$$

Gradient

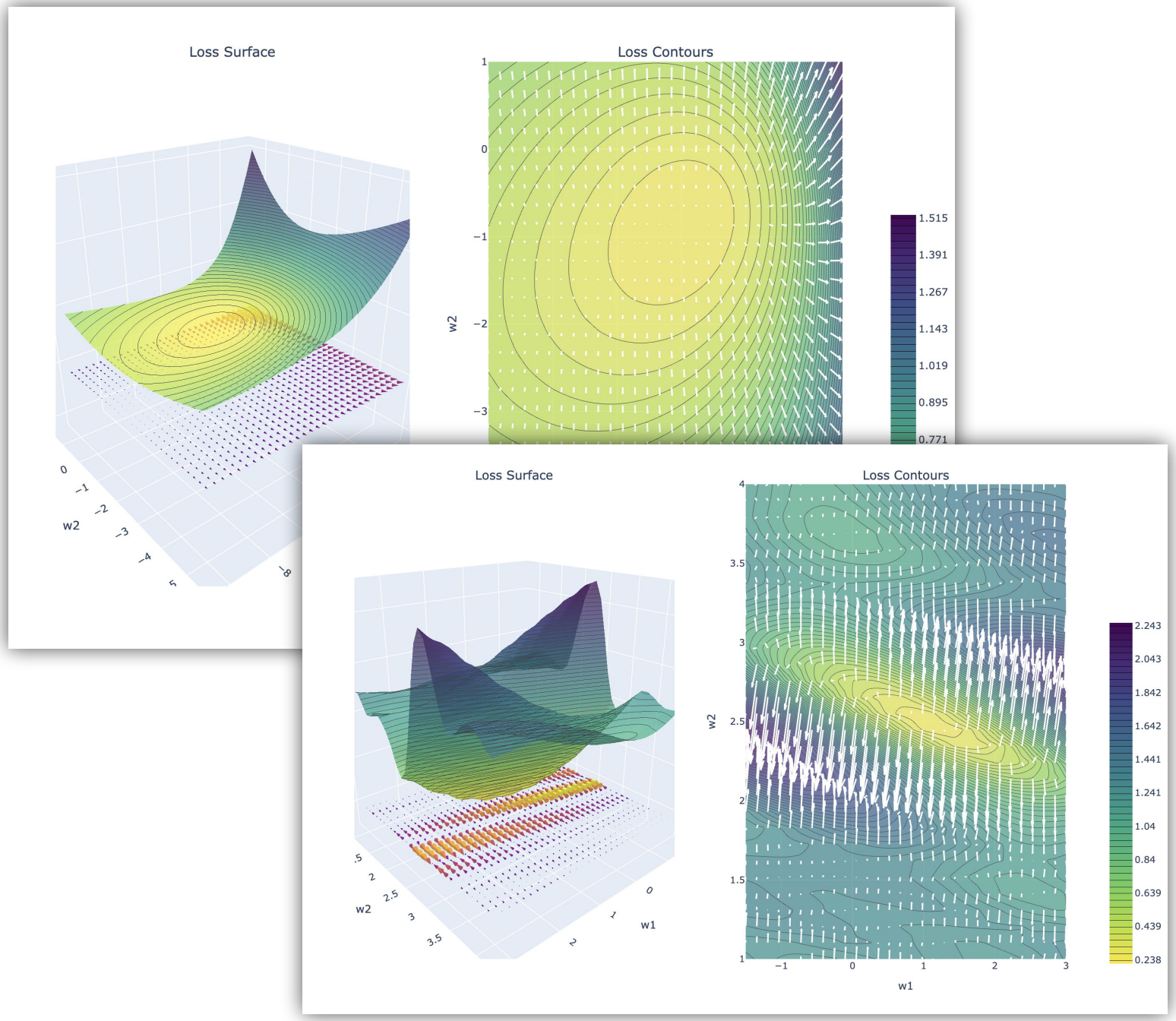
The gradient points in the **direction of greatest ascent**.

Zero Gradient at $w = (1,2)$

Plots in demo notebook.

Demo

The Gradient of the Error Function



If w is a d-dimensional vector and E(w) is a scalar-valued function, what is the dimensionality of the gradient $\nabla E(w)$ with respect to w?

Common Confusion

1846379

$$E(w) = (w_0 - 1)^2 + (w_1 - 2)^2 + 1$$

- The gradient does not lie on the surface of the function.

Not the gradient!

Example: $E: \mathbb{R}^2 \to \mathbb{R}$

The gradient $\nabla E: \mathbb{R}^2 \to \mathbb{R}^2$ is **2-dimensional (not 3)**

The gradient is in the **domain** of the function.

Stationary Points and Zero Gradient

1846379

Anywhere the gradient is zero is a **stationary point.**

- Gradient is zero at local minima but zero gradient does not imply a minima

$$E_A[w] = (w_0 - 1)^2 + (w_1 - 2)^2 + 1$$

Minimum

$$E_B[w] = -(w_0 - 1)^2 - (w_1 - 2)^2 + 12$$

Maximum

$$E_C[w] = -(w_0 - 1)^2 + (w_1 - 2)^2 + 1$$

Saddle Point

Plots in demo notebook.

Analytical Optimization using Gradients

1846379

- In some situations, (e.g., **ordinary least squares**) we can find the stationary points by solving:

$$\nabla E(w) = 0$$

- For these problems we often know that there is one stationary point and that it is the minima.

However, for many machine learning applications (e.g., logistic regression and neural networks) **we cannot solve for the stationary points analytically**.

Iterative optimization

Gradient Descent

Iterative Optimization Algorithms

1846379

Iterative optimization algorithms start with an **initial estimate** $\mathbf{w}^{(0)}$ and then **iteratively refine** that estimate:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}^{(\tau-1)}$$

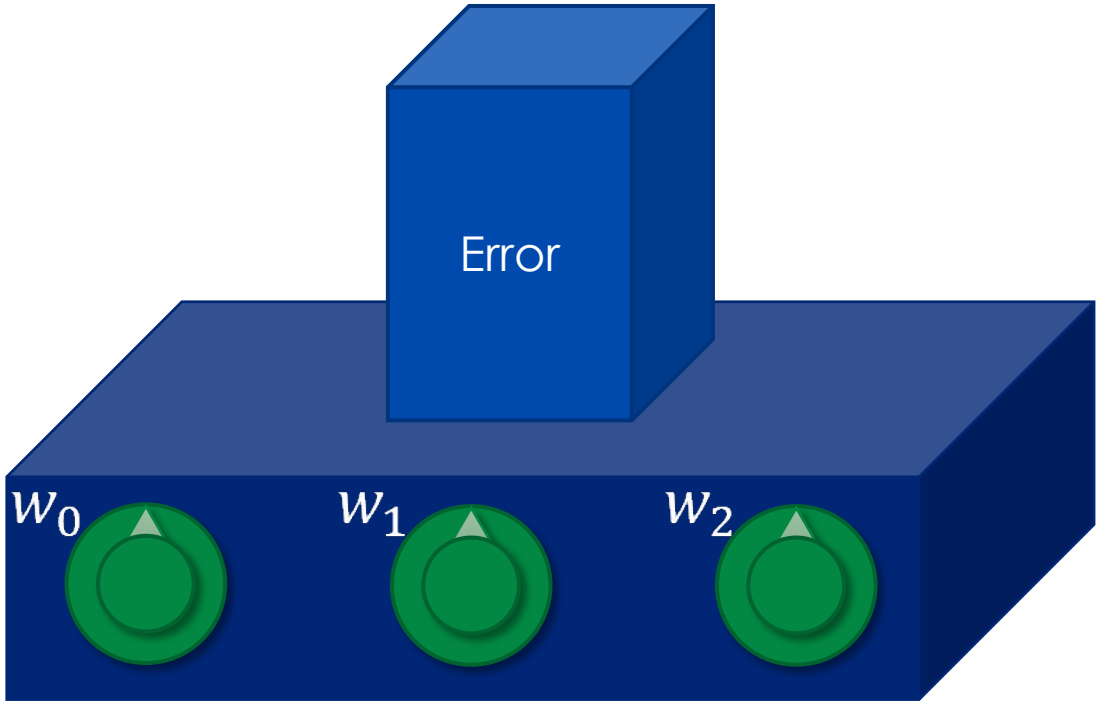
- Iteration τ
- New Estimate
- Old Estimate
- Update (Based on **Gradient**)

until **stopping criterion** is achieved.

The choice of initial estimate $\mathbf{w}^{(0)}$, how the update $\Delta\mathbf{w}^{(\tau-1)}$ is computed, and the **stopping criterion** define the **iterative optimization algorithm**.

Intuition

1846379

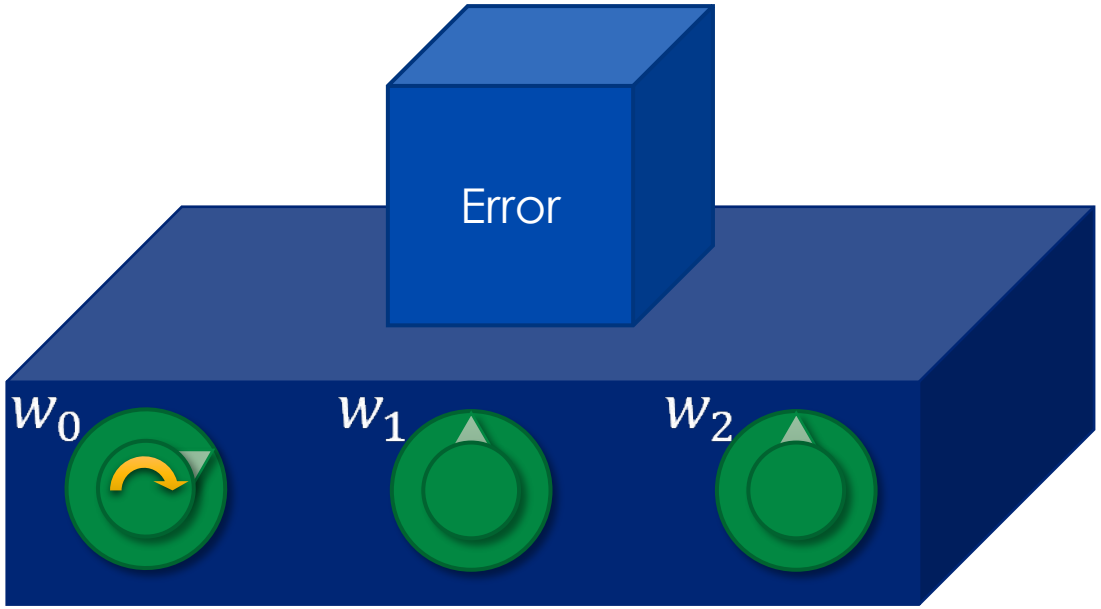


Goal: Minimize the loss by turning the knobs.

Try the loss game (its free)!

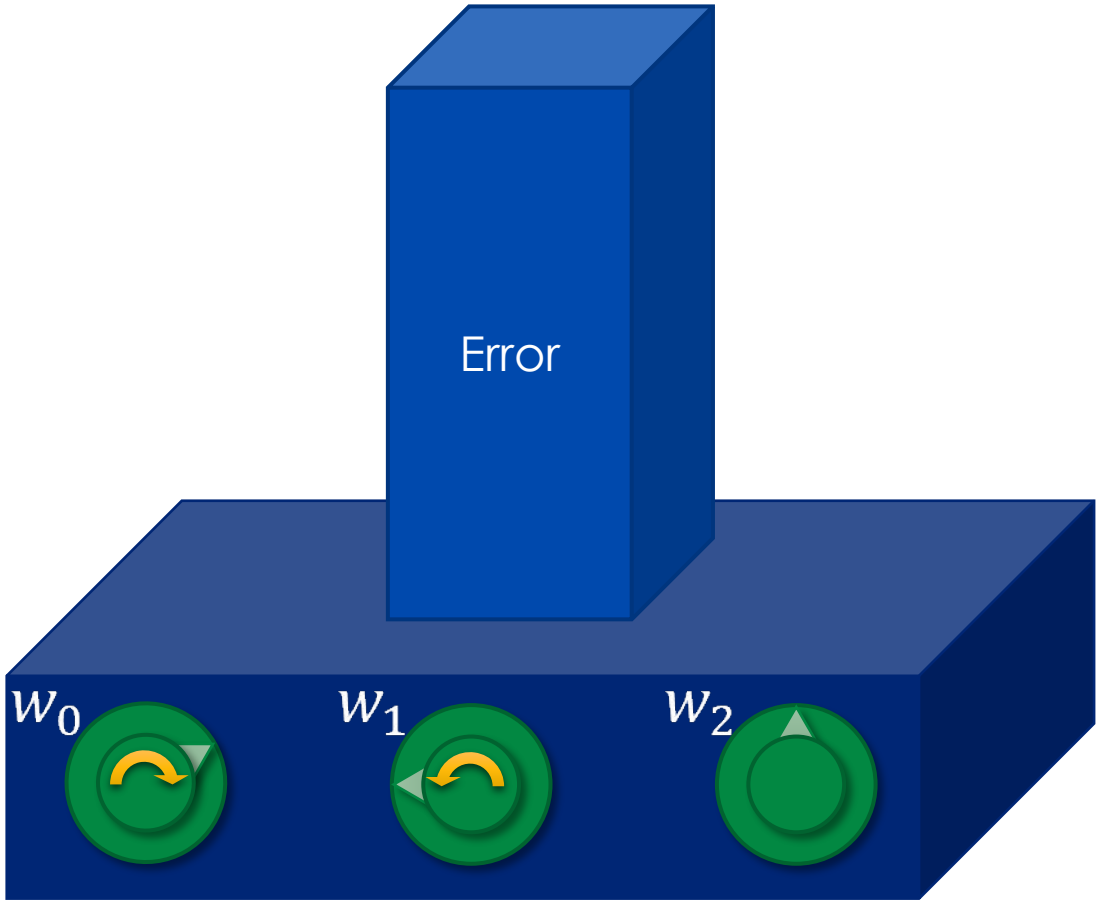
Intuition

1846379



Try the loss game (its free)!

Intuition

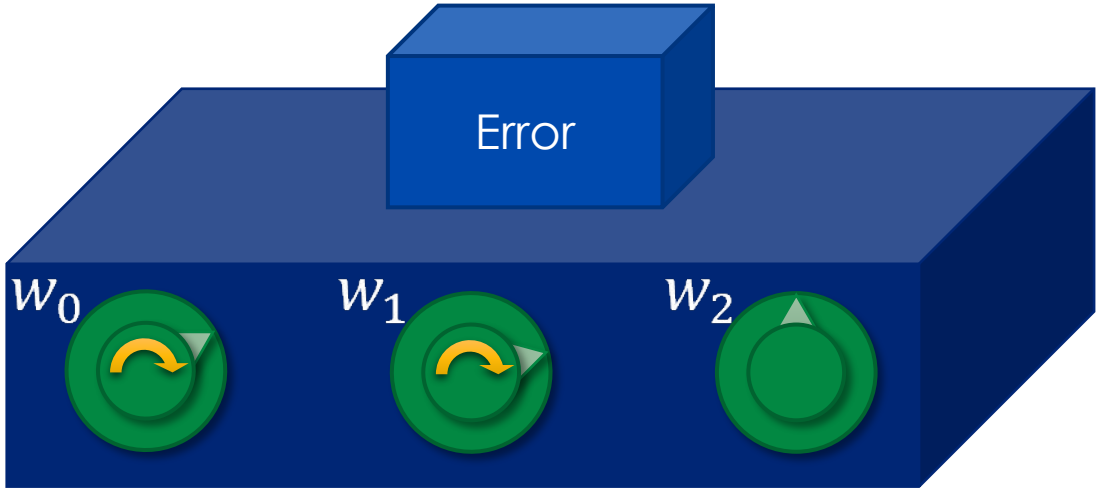


1846379

Try the loss game (its free)!

Intuition

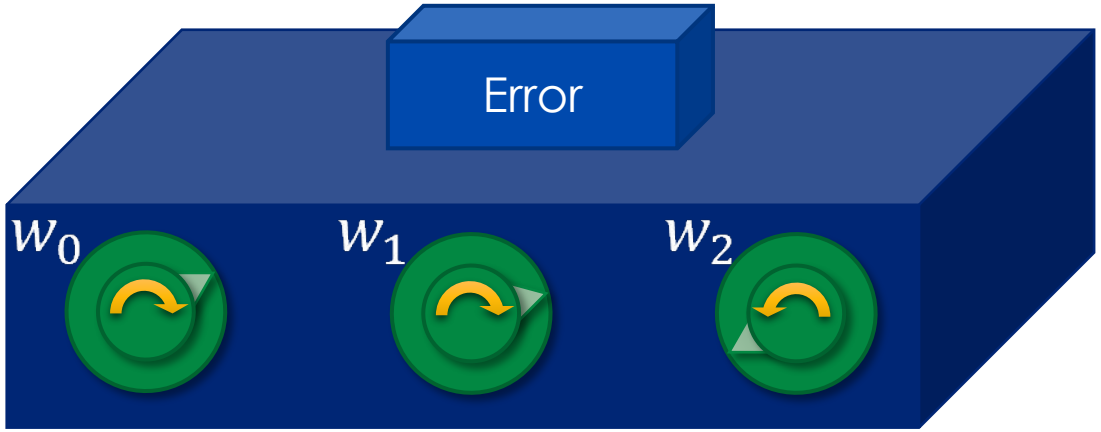
1846379



Try the loss game (its free)!

Intuition

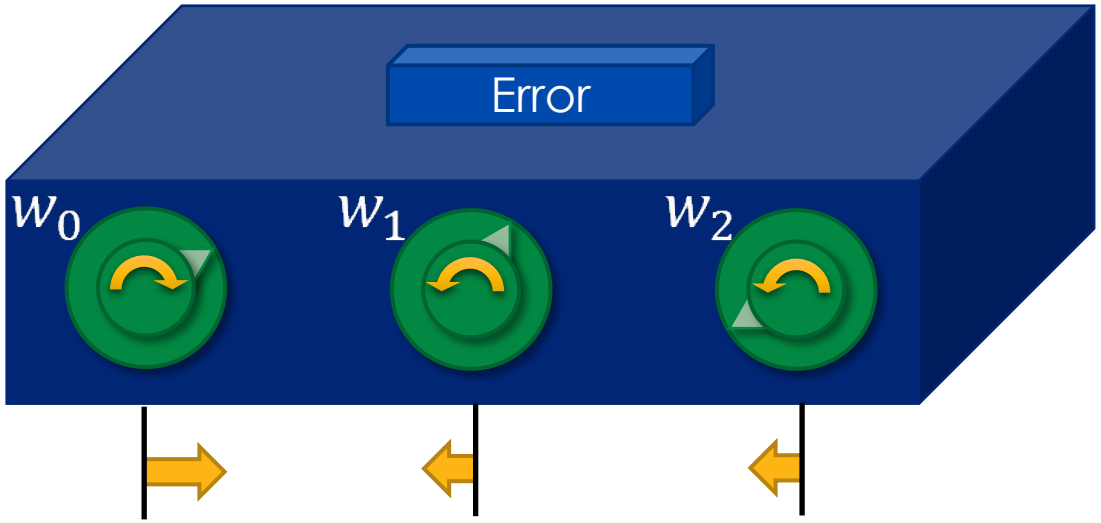
1846379



Try the loss game (its free)!

Intuition

1846379



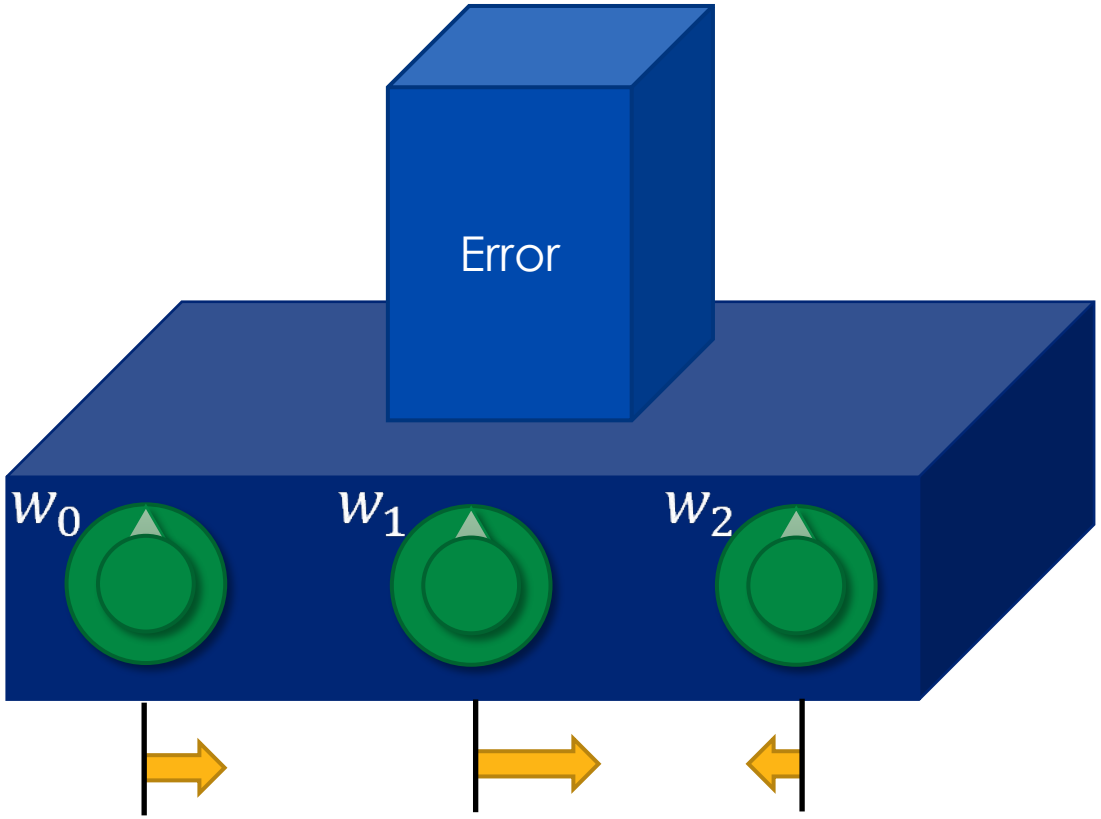
What if we knew which way to turn the knob and an idea of how far?

This is the Gradient!

Try the loss game (its free)!

Intuition

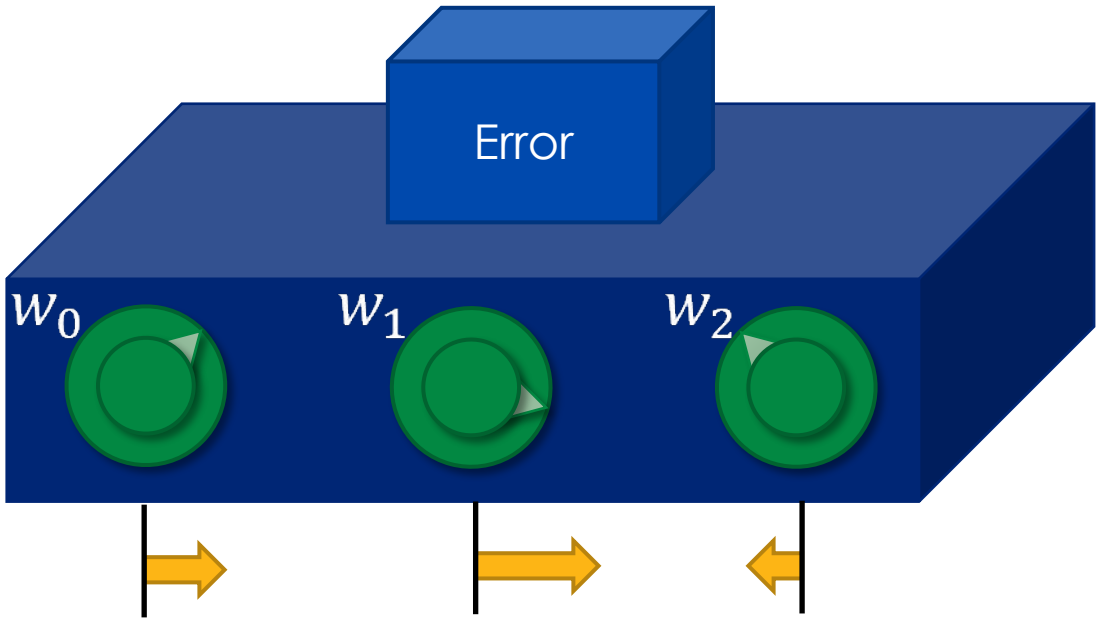
1846379



Try the loss game (its free)!

Intuition

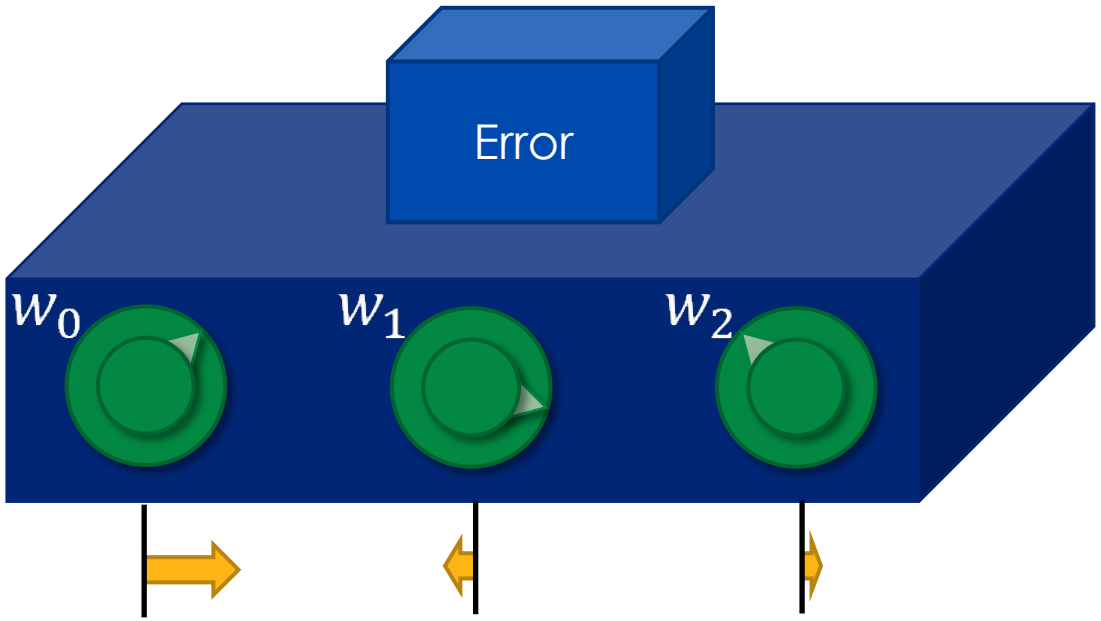
1846379



Try the loss game (its free)!

Intuition

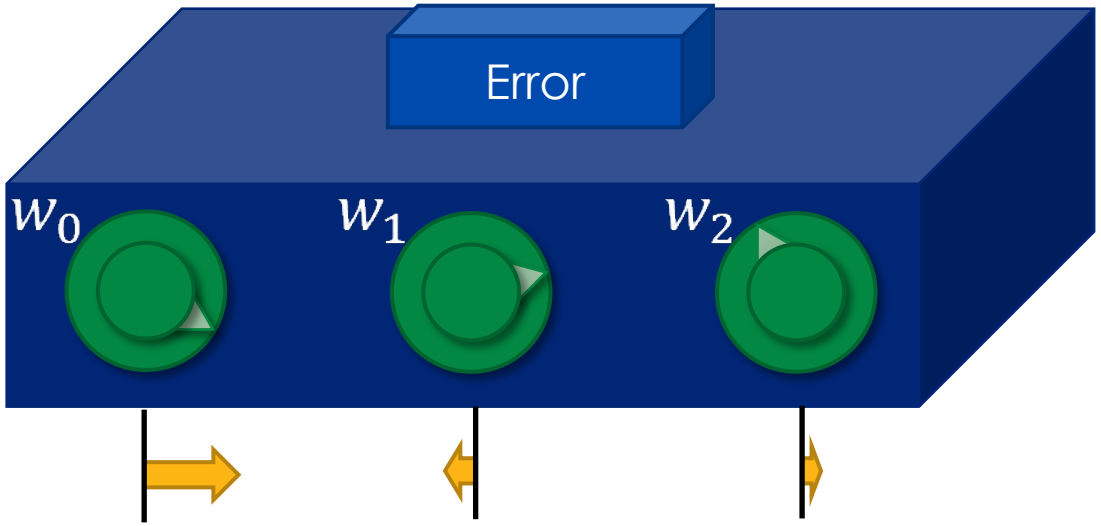
1846379



Try the loss game (its free)!

Intuition

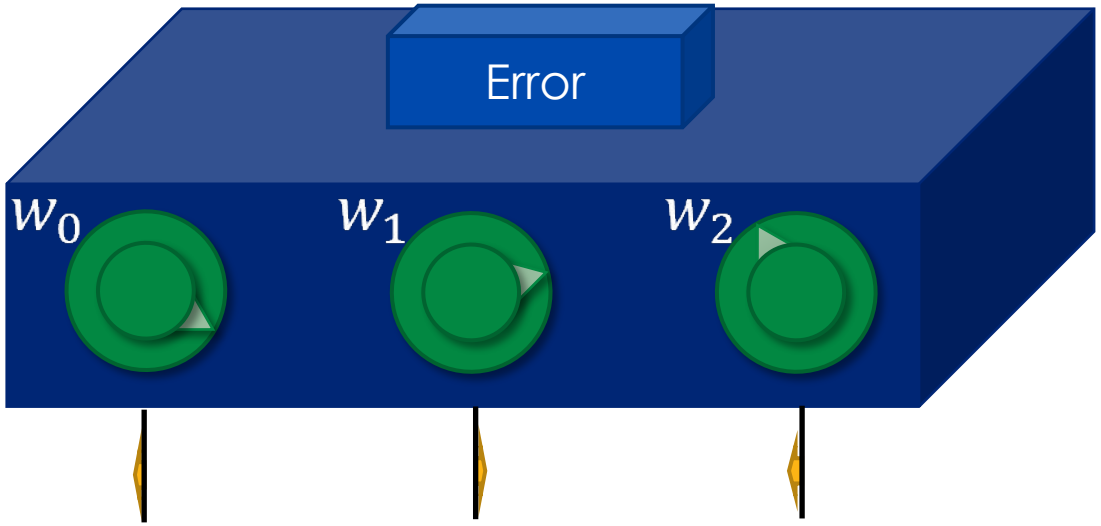
1846379



Try the loss game (its free)!

Intuition

1846379

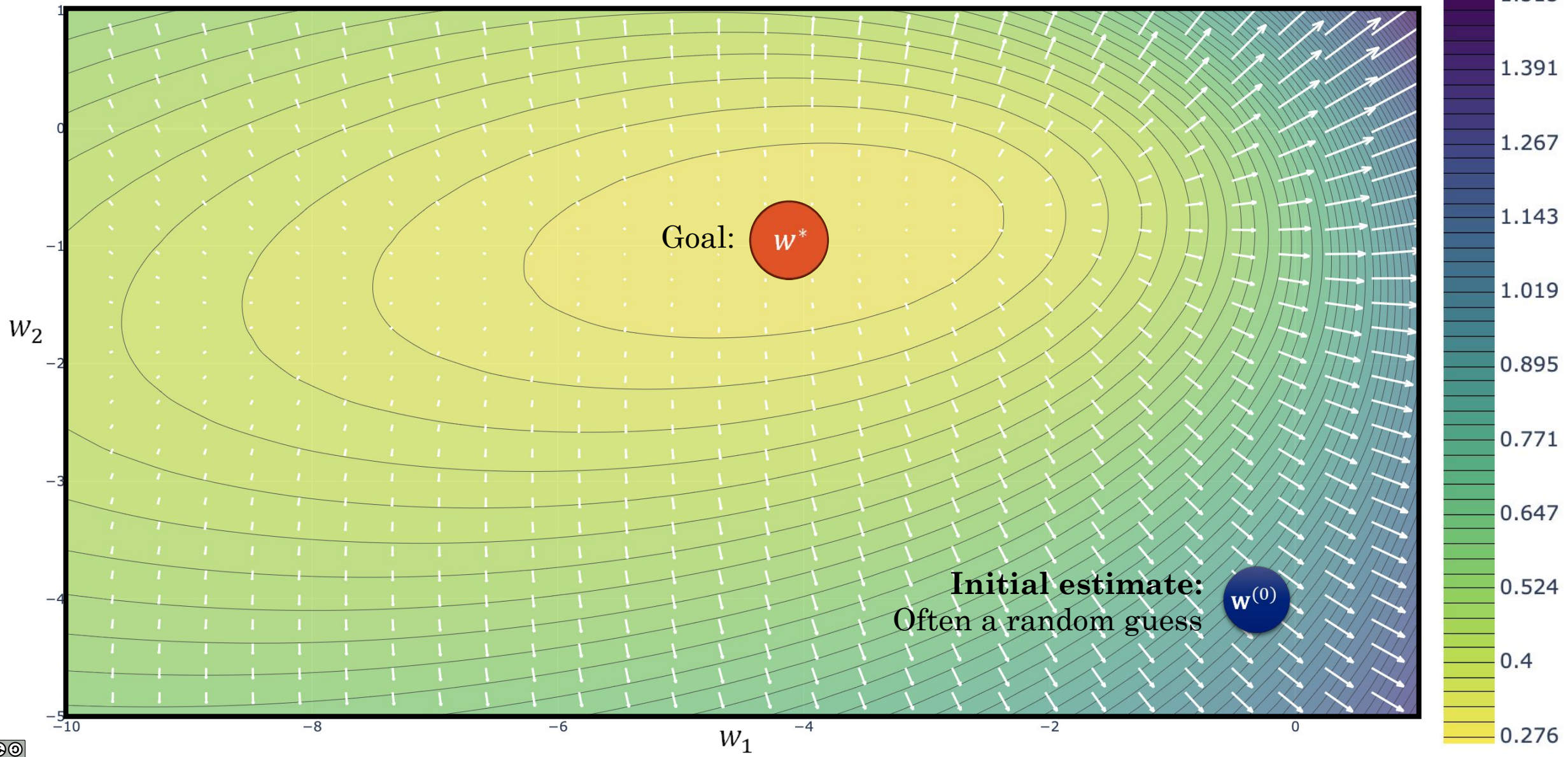


This is Gradient descent!

Try the loss game (its free)!

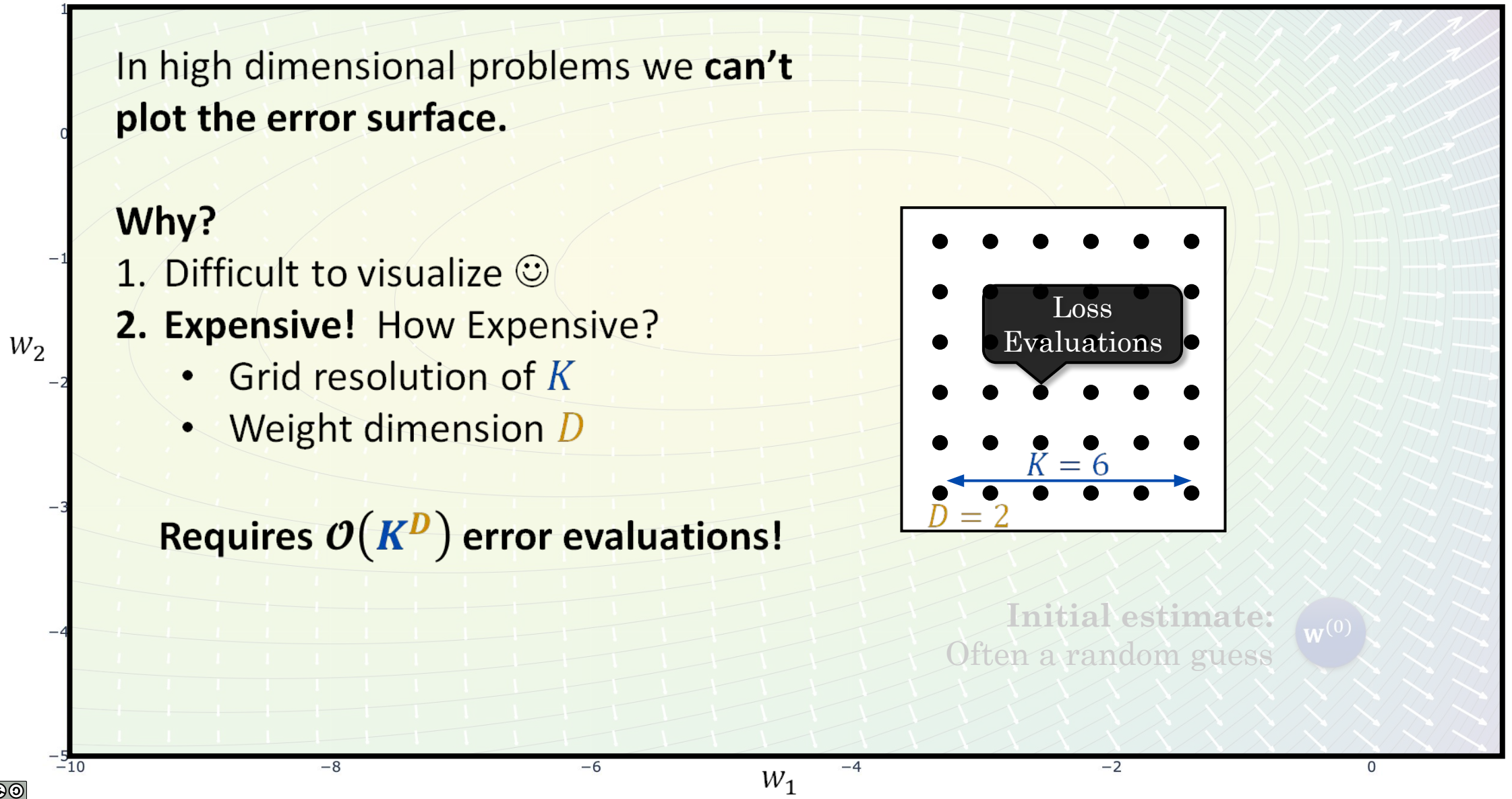
Gradient Descent Intuition

1846379



Gradient Descent Intuition

1846379



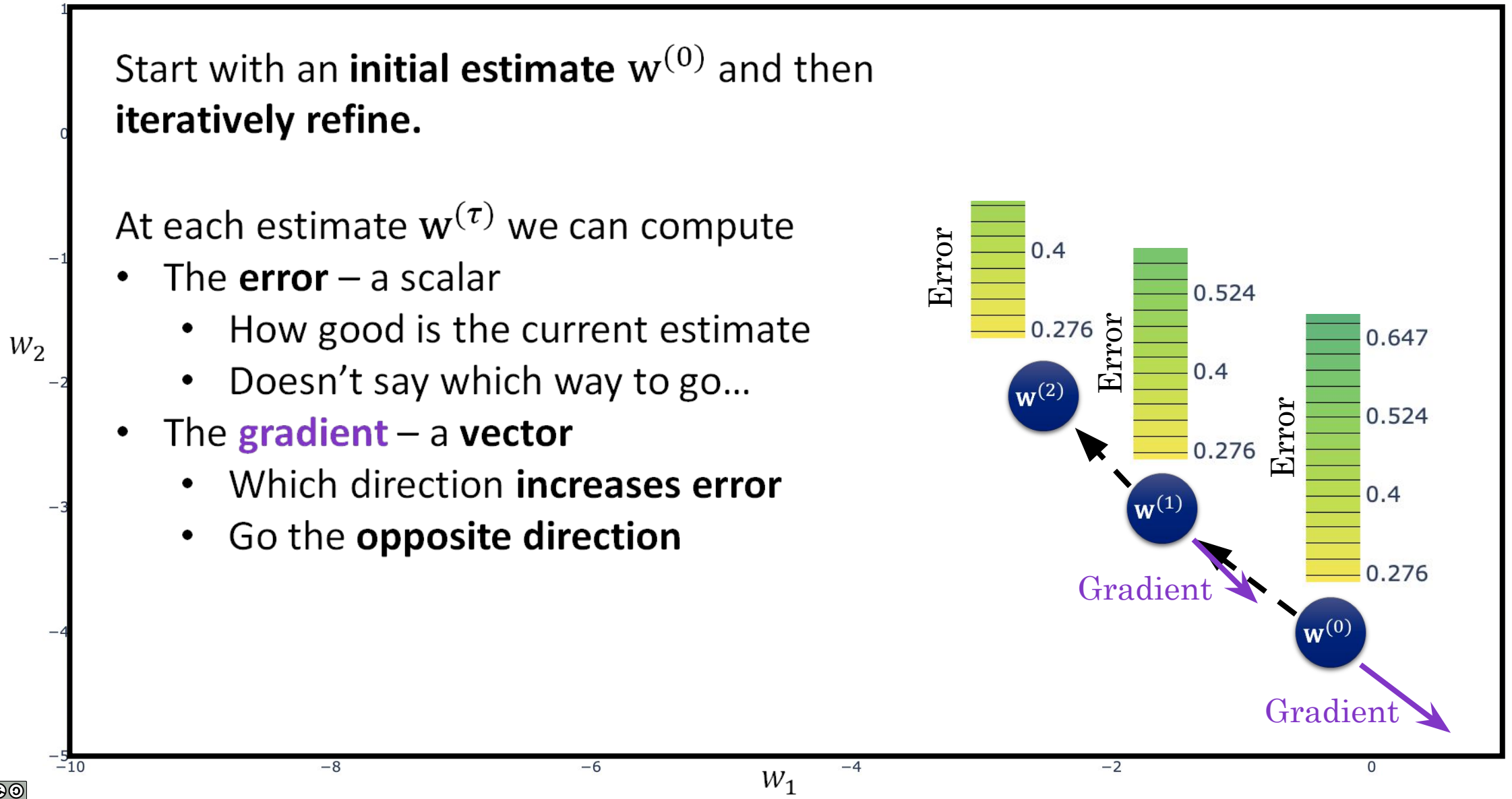
Gradient Descent Intuition

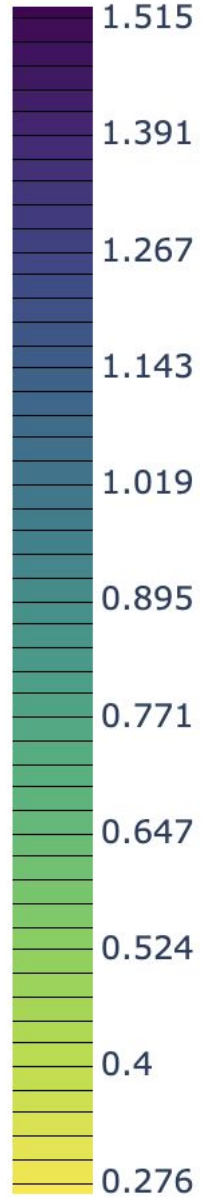
1846379

Start with an **initial estimate** $\mathbf{w}^{(0)}$ and then **iteratively refine.**

At each estimate $\mathbf{w}^{(\tau)}$ we can compute

- The **error** – a scalar
  - How good is the current estimate
  - Doesn't say which way to go…
- The **gradient** – a **vector**
  - Which direction **increases error**
  - Go the **opposite direction**





The (Batch) **Gradient Descent** Alg.

Update the weights by moving in the **opposite direction** of the gradient:

$$\mathbf{w}^{(0)} = \text{Initialize}()$$
$$\text{for } \tau \text{ in range}(1, \tau_{\max}):$$
$$\quad \mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E\left(\mathbf{w}^{(\tau-1)}\right)$$
$$\quad \text{if } \text{Converged}\left(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon\right): \text{terminate}$$

$\text{Initialize}()$ typically a small random value (more on this later)

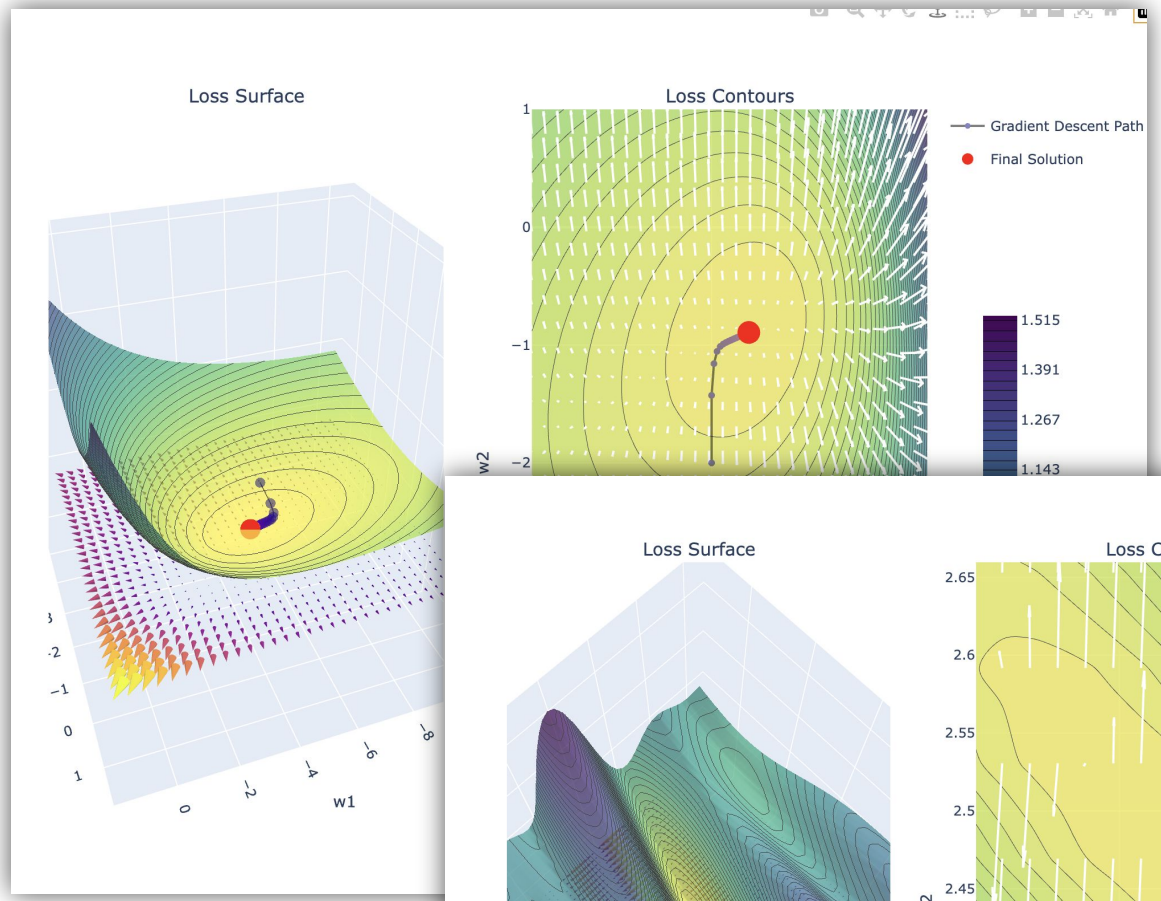
$\text{Converged}\left(\mathbf{w}^{(\tau)}, \mathbf{w}^{(\tau-1)}, \epsilon\right)$ stopping condition (e.g., ϵ change in $\Delta\mathbf{w}^{(\tau-1)}$)

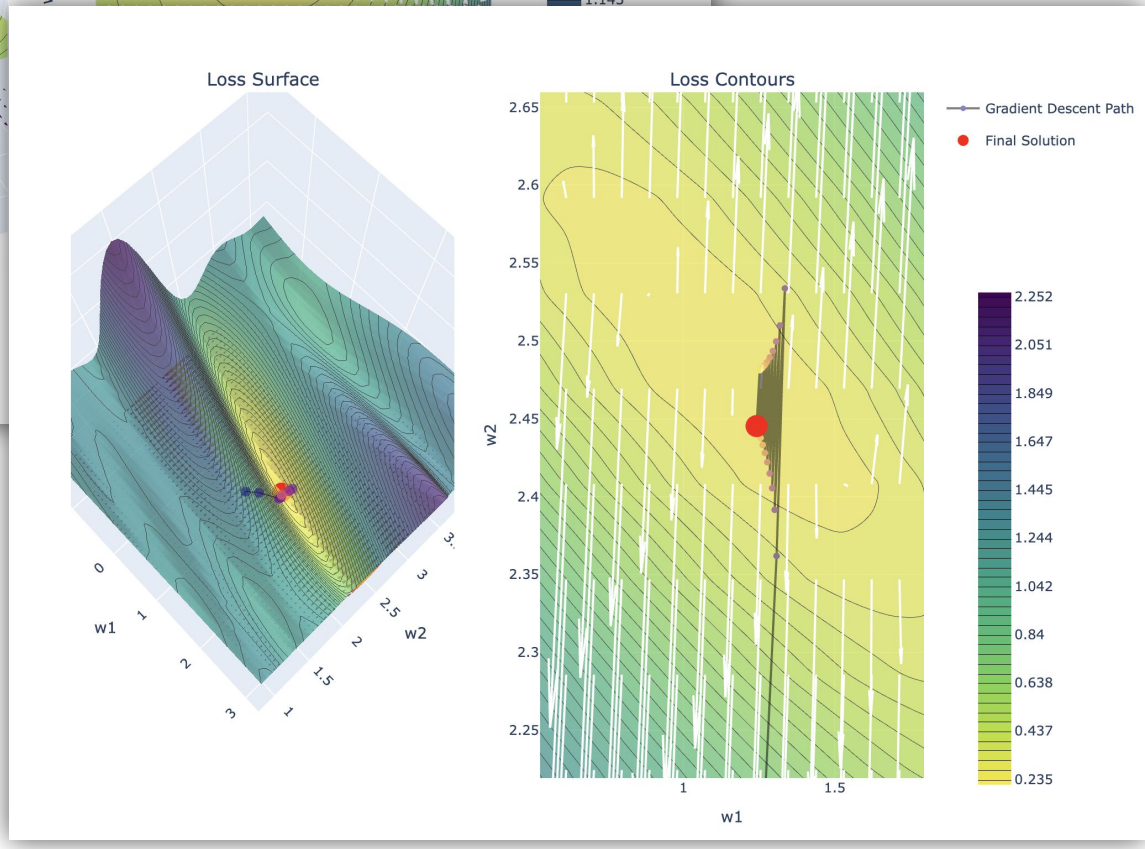
Hyper Parameters:

- η is the learning rate (typically a small value $0 < \eta < 1$, more on this later)
- $\tau_{\max}$ is the maximum number of iterations

Demo

Batch Gradient Descent





Loss (Error) Curves

1846379

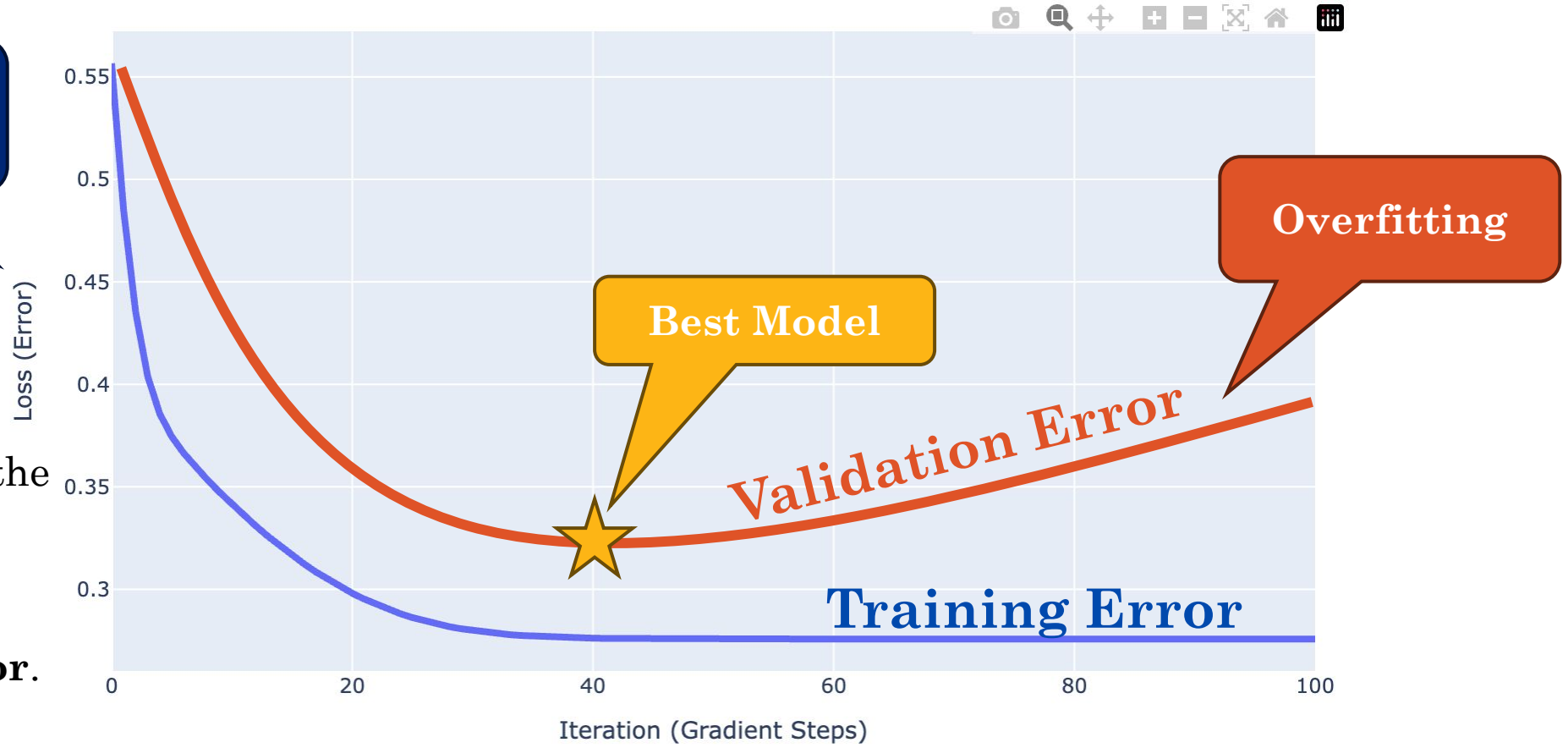
The **loss** or **error curve plots** the error as a function of the number of **iterations** (**steps**) of gradient descent.

Training and **Validation** Error

Validation curves are typically **above** the training curves.

You can **overoptimize** the objective (**overfit**).

Choose parameters with **lowest validation error**.



Convergence Assuming Quadratic Approximation

Assuming a simple quadratic equation (e.g., x^2) is gradient descent guaranteed to converge?

Quadratic Approximations

1846379

- The 2nd order (quadratic) Taylor approximation of a function function $f: \mathbb{R} \to \mathbb{R}$ evaluated at $\hat{x}$ is:

$$\tilde{f}(x) = f(\hat{x}) + \left(\frac{\partial}{\partial x} f(x)\Big|_{x=\hat{x}}\right)(x - \hat{x}) + \frac{1}{2}\left(\frac{\partial^2}{\partial x^2} f(x)\Big|_{x=\hat{x}}\right)(x - \hat{x})^2$$

For example: $f(x) = \sin(x)$

$$\tilde{f}_{\hat{x}=1}(x) = \sin(1) + \cos(1)\,(x-1) + \frac{1}{2}(-\sin(1))(x-1)^2$$

$$\tilde{f}_{\hat{x}=5}(x) = \sin(5) + \cos(5)\,(x-5) + \frac{1}{2}(-\sin(5))(x-5)^2$$

Taylor Expansion of the Error Surface

The 2nd-order Taylor expansion of the **error surface** at $\hat{w}$:

$$\tilde{E}_{\hat{w}}(w) = E(\hat{w}) + (w - \hat{w})^T b + \frac{1}{2}(w - \hat{w})^T H (w - \hat{w})$$

- $b = \nabla E(\hat{w})$ is the **Gradient** of E evaluated at $\hat{w}$ (vector)
- $H = \nabla\nabla E(\hat{w})$ is the **Hessian** of E evaluated at $\hat{w}$ (a matrix)

Matrix Equivalent of the Second Derivative

Hessians (the 2nd derivative)

1846379

The Hessian of a **scalar-valued function** $E: \mathbb{R}^D \to \mathbb{R}$ is the **matrix-valued function** $\nabla^2 E: \mathbb{R}^D \to \mathbb{R}^{D \times D}$ of the 2nd partial derivatives of f

$$H = \nabla^2 E = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} E(x) & \dots & \frac{\partial^2}{\partial x_1 x_D} E(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_D x_1} E(x) & \dots & \frac{\partial^2}{\partial x_D^2} E(x) \end{bmatrix}$$

The Hessian

- describes the local **curvature** of E
- determines the **critical point type** (minima, maxima, or saddle)

Stopped Here

Will return to later material in the next lecture.

Hessian Exercise

1846379

Consider the function:

$$E(w) = (w_0 - 1)^2 + (w_1 - 2)^2 + 1$$

Then the gradient is: $\nabla E(w) = \begin{bmatrix} 2(w_0 - 1) \\ 2(w_1 - 2) \end{bmatrix}$

The Hessian is:

$$\mathrm{H(w)} = \begin{bmatrix} \frac{\partial^2}{\partial w_0^2} E(w) & \frac{\partial^2}{\partial w_0 \partial w_1} E(w) \\ \frac{\partial^2}{\partial w_0 \partial w_1} E(w) & \frac{\partial^2}{\partial w_1^2} E(w) \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Hessian

$$\nabla^2 E = \begin{bmatrix} \frac{\partial^2}{\partial x_1^2} E(x) & \cdots & \frac{\partial^2}{\partial x_1 x_D} E(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2}{\partial x_D x_1} E(x) & \cdots & \frac{\partial^2}{\partial x_D^2} E(x) \end{bmatrix}$$

Plots in demo notebook.

Taylor Expansion of the Error Surface

The 2nd-order Taylor expansion of the error surface at $\hat{w}$:

$$\tilde{E}_{\hat{w}}(w) = E(\hat{w}) + (w - \hat{w})^T b + \frac{1}{2}(w - \hat{w})^T H (w - \hat{w})$$

- $b = \nabla E(\hat{w})$ is the **Gradient** of E evaluated at $\hat{w}$ (vector)
- $H = \nabla\nabla E(\hat{w})$ is the **Hessian** of E evaluated at $\hat{w}$ (a matrix)

If $\hat{w}$ is a **stationary point** (w^*) then $\nabla E(w^*) = 0 = b$

$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2}(w - w^*)^T H (w - w^*)$$

Use a change of variables to better understand this part.

Eigen Decomposition of the Hessian

We can construct the **Eigen decomposition of the Hessian**

$$Hu_i = \lambda_i u_i$$

- u_i are the complete set of **orthonormal eigen vectors:** $\forall_{i,j}: u_i^T u_i = \delta_{ij}$
- λ_i are the real-valued **eigen values**

Then we apply a **change of variables**:

$$w - w^* = \sum_i \alpha_i u_i \quad \text{where} \quad \alpha_i = (w - w^*)^T u_i$$

This allows us to rewrite:

$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2}(w - w^*)^T H (w - w^*) = E(w^*) + \frac{1}{2}\sum_i \alpha_i^2 \lambda_i$$

Show it!

- u_i are the complete set of **orthonormal eigen vectors:** $\forall_{i,j}\colon\ u_i^T u_i = \delta_{ij}$
- λ_i are the real-valued **eigen values**

1846379

Then we apply a **change of variables**:

$$w - w^* = \sum_i \alpha_i u_i \quad \text{where} \quad \alpha_i = (w - w^*)^T u_i$$

This allows us to rewrite:

$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2}(w - w^*)^T H (w - w^*) = E(w^*) + \frac{1}{2}\sum_i \alpha_i^2 \lambda_i$$

Show it!

Derivation:

$$(w - w^*)^T H (w - w^*) = \left(\sum_i \alpha_i u_i\right)^T H \left(\sum_i \alpha_i u_i\right) = \left(\sum_i \alpha_i u_i\right)^T \left(\sum_i \alpha_i H u_i\right)$$

$$= \left(\sum_i \alpha_i u_i\right)^T \left(\sum_i \alpha_i \lambda_i u_i\right) = \sum_{ij} \alpha_i \alpha_j u_i^T u_j \lambda_j = \sum_{ij} \alpha_i \alpha_j \delta_{ij} \lambda_j = \sum_i \alpha_i^2 \lambda_i$$

Eigenvector Defn.

Orthonormality

Taylor Expansion at the Stationary Pts.

If $\hat{w}$ is a **stationary point** (w^*) (where $b = \nabla E|_{w=\hat{w}} = 0$)

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2}\sum_i \alpha_i^2 \lambda_i$$

At $\alpha = 0$ we get the error at the critical point $E(w^*)$.

- If we move in **direction *i*** (corresponding to u_i) then α_i^2 **increases.**
  - The change in error then depends on the sign of λ_i

Therefore, if the **eigenvalues are**

- **all positive** then w^* is a **local minimum.**
- **all negative** then w^* is a **local maximum.**
- **mixed** then w^* is a **saddle point.**

Stationary Points and the Hessian

1846379

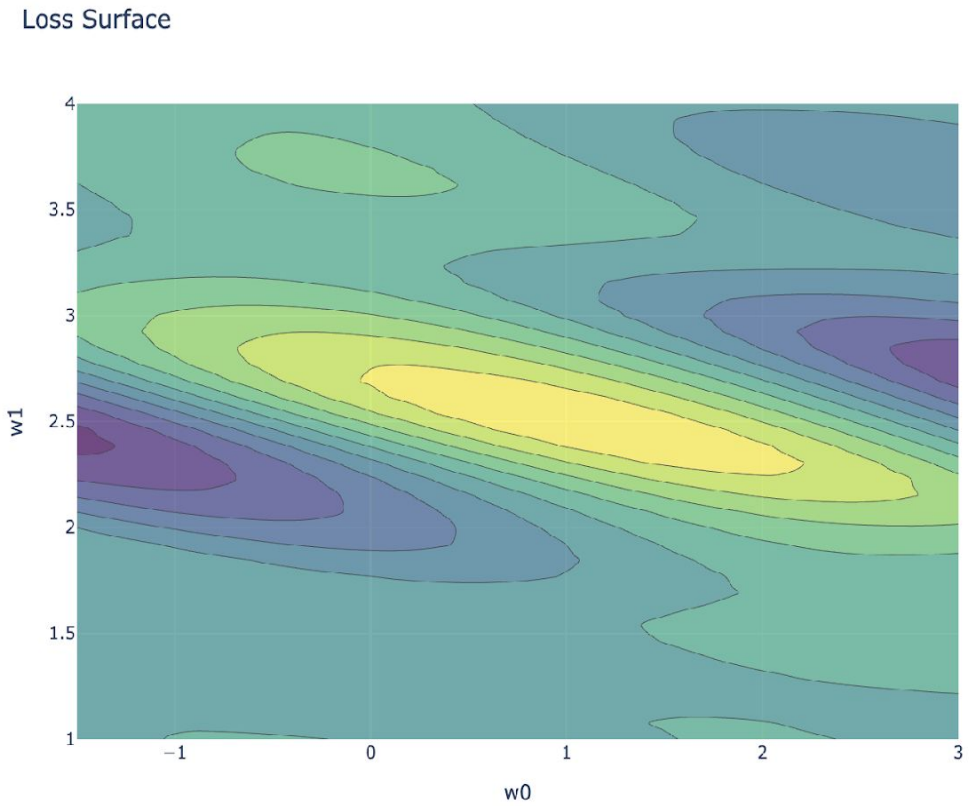
Eigenvalues of the Hessian determine the curvature at the critical points

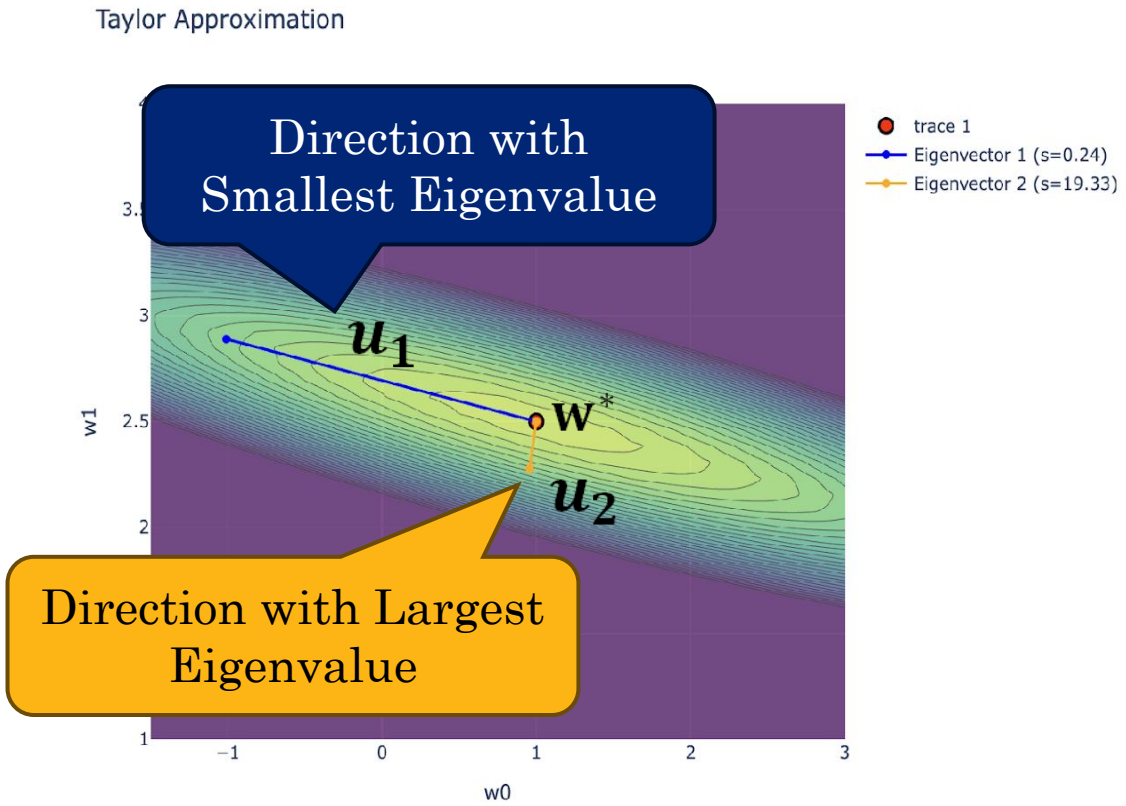
Minimum	**Maximum**	**Saddle Point**

Eigenvector of the Hessian

1846379

Elliptical contours of constant error align with the eigenvectors of the Hessian matrix.

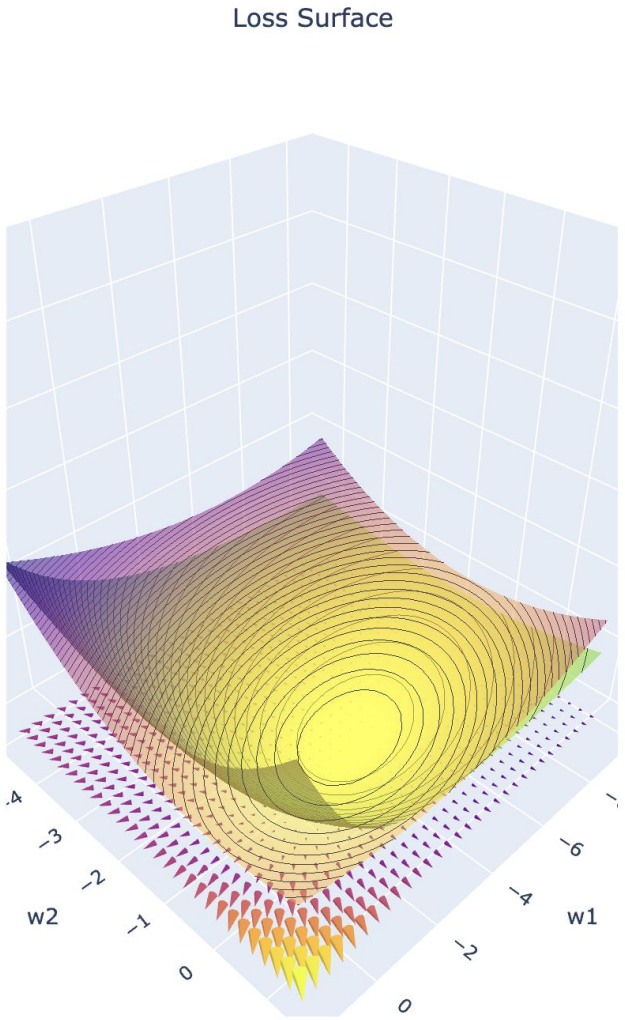


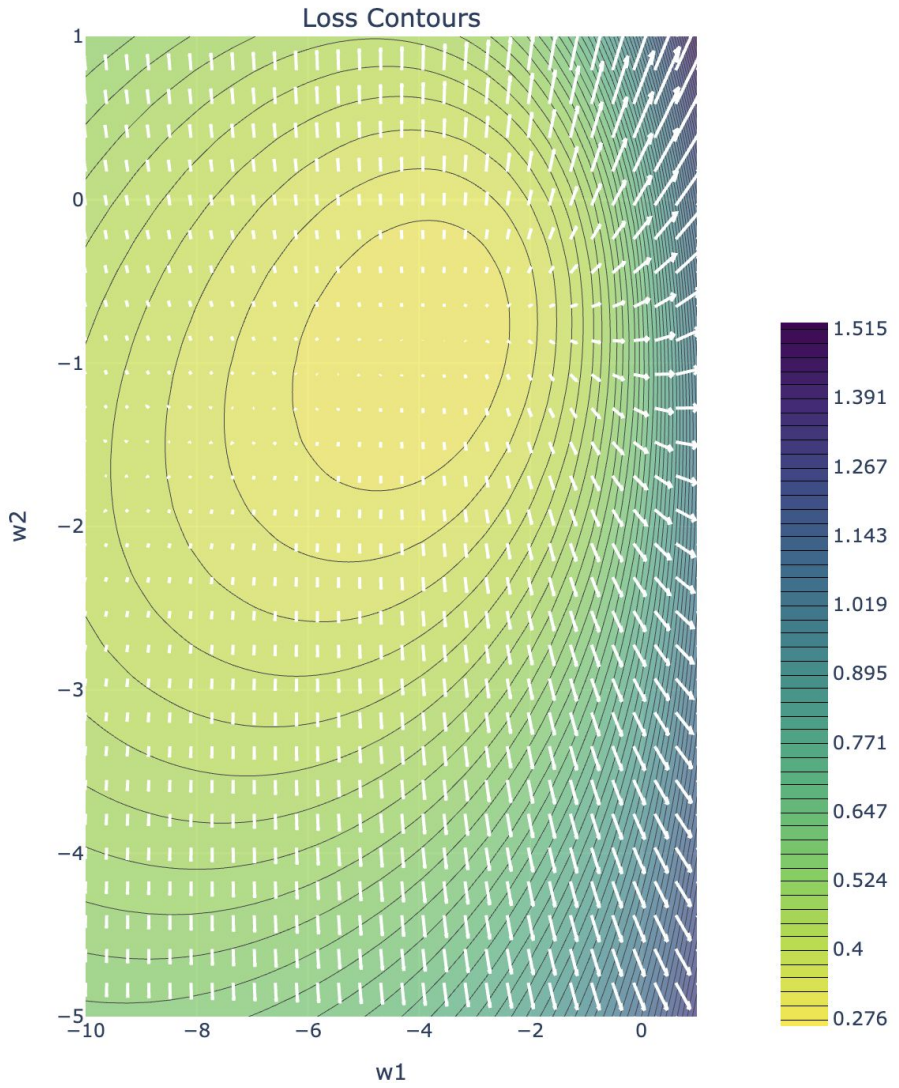


Plots in demo notebook.

Demo

Taylor Expansions and the Hessian





Recap: Quadratic Approx. Error

1846379

We constructed the **2nd-order Taylor expansion** of the error surface at the **stationary point** w^* (where $b = \nabla E|_{w=w^*} = 0$):

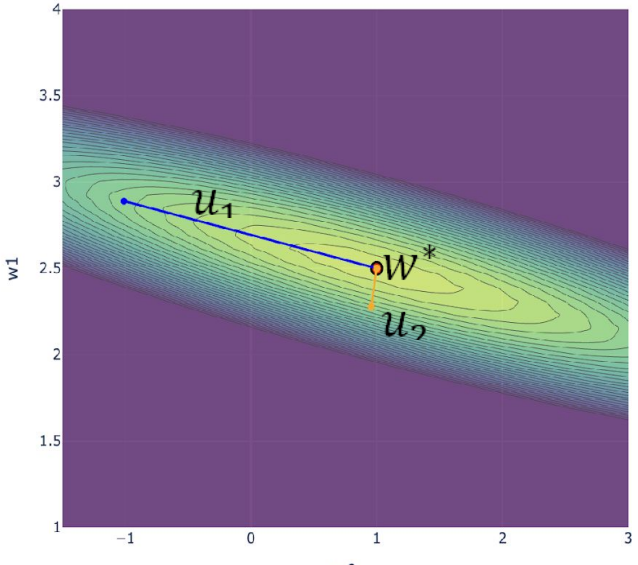
$$\tilde{E}_{w^*}(w) = E(w^*) + \frac{1}{2}(w - w^*)^T H (w - w^*)$$

Using the **Eigen-decomposition of the Hessian:** $Hu_i = \lambda_i u_i$ we applied a change of variables $\alpha_i = (w - w^*)^T u_i$ to obtain:

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2}\sum_i \alpha_i^2 \lambda_i$$

- **This is minimized at $\alpha = 0$**

We can **evaluate gradient descent** on this simple quadratic approximation to better understand the convergence properties.



Grad. Descent on the Quadratic Approx.

Quadratic Approx. Error

$$\tilde{E}_{w^*}(\alpha) = E(w^*) + \frac{1}{2}\sum_i \alpha_i^2 \lambda_i$$

- Compute the **gradient** with respect to α_i:

$$\frac{\partial}{\partial \alpha_i} \tilde{E}_{w^*}(\alpha) = \alpha_i \lambda_i$$

Which gives the **gradient descent update**:

$$\alpha_i^{(\tau)} = \alpha_i^{(\tau-1)} - \eta\left(\alpha_i^{(\tau-1)} \lambda_i\right) = (1 - \eta\lambda_i)\alpha_i^{(\tau-1)}$$

If we **iteratively** apply the gradient descent update, we obtain:

$$\alpha_i^{(1)} = (1 - \eta\lambda_i)\alpha_i^{(0)} \rightarrow \alpha_i^{(2)} = (1 - \eta\lambda_i)\left((1 - \eta\lambda_i)\alpha_i^{(0)}\right) \rightarrow \ldots \rightarrow$$

$$\alpha_i^{(\tau)} = (1 - \eta\lambda_i)^{\tau}\alpha_i^{(0)}$$

Grad. Descent on the Quadratic Approx.

Assuming a **quadratic approximation** to the error function around the minima, the τ^{th} iteration of gradient descent we have:

$$\alpha_i^{(\tau)} = (1 - \eta\lambda_i)^{\tau}\alpha_i^{(0)}$$

and the **error is minimized when $\boldsymbol{\alpha_i = 0}$.**

To ensure $\alpha_i^{(\tau)} \to 0$ as $\tau \to \infty$ the learning rate should satisfy:

$$\boldsymbol{|1 - \eta\lambda_i| < 1}$$ **(Convergence Condition)**

- Smaller $\boldsymbol{|1 - \eta\lambda_i|}$ is better (converges faster)
- Negative $\boldsymbol{1 - \eta\lambda_i} < 0$ oscillatory behavior

Learning Rate Impact on Convergence

To maximize the **progress on each step** of gradient descent we want to **make η as large as possible (take bigger steps)**

- We want $(1-\eta\lambda_i)^\tau \to 0$ quickly as τ increases (larger η ➔ smaller $1-\eta\lambda_i$).
- However, to **ensure convergence we require $|1-\eta\lambda_i| < 1$ for all λ_i**

Let $\lambda_{\min}$ and $\lambda_{\max}$ be the smallest and largest **eigenvalues** of the Hessian

- Then $\eta < \frac{2}{\lambda_{\max}}$ because if $\eta \geq \frac{2}{\lambda_{\max}}$ then $|1-\eta\lambda_{\max}| \geq 1$

The **slowest converging dimension correspond to λ_{min}** so the smallest we can make $(1-\eta\lambda_i)$:

$$\left(1-\frac{2}{\lambda_{\max}}\lambda_{\min}\right) < (1-\eta\lambda_i)$$

Condition Number and Loss Surface

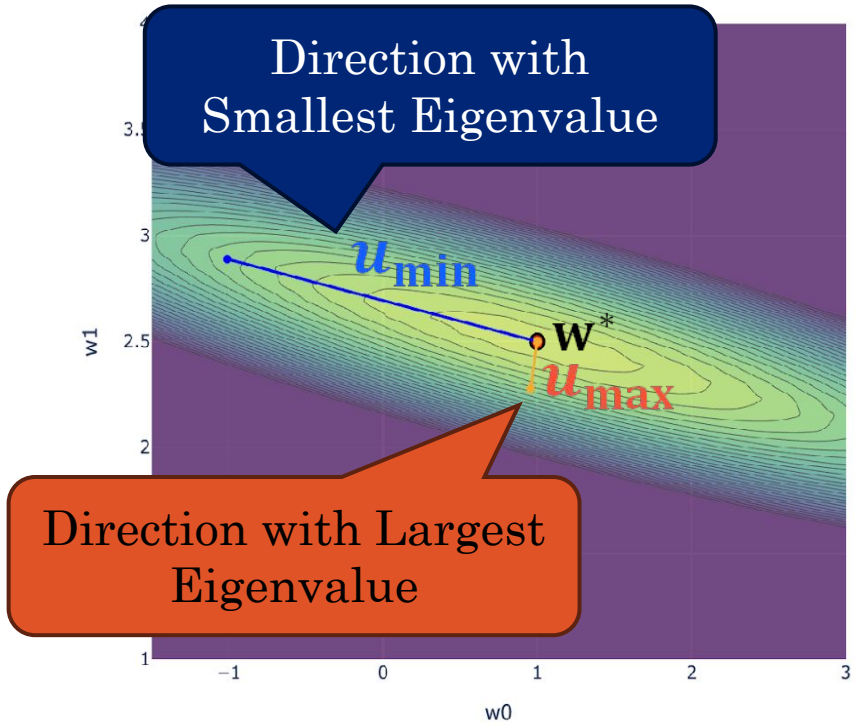
- The slowest converging dimension (i) is the dimension corresponding to λ_{min} and its convergence to 0 is lower bounded by:

$$\left(1 - \frac{2}{\lambda_{\max}} \lambda_{\min}\right)^{\tau} \alpha_i^{(0)}$$

The ratio of $\frac{\lambda_{\min}}{\lambda_{\max}}$ is the **condition number**.

- A problem is said to be **well conditioned** if it has a **large condition number.**

Gradient descent converges quickly on **well conditioned problems**.



Issues with Gradient Descent (GD)

GD converges slowly when the **error surface is relatively flat**:

Step size
(learning rate η) is
too small.

GD oscillates when the error surface is **poorly conditioned**

Step size
(learning rate η) is
too big.

Demo

Batch Gradient Descent

Momentum

Momentum can be used improve convergence in these settings by **accumulating common update directions**

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E\left(\mathbf{w}^{(\tau-1)}\right) + \mu \Delta \mathbf{w}^{(\tau-2)}$$

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta \mathbf{w}^{(\tau-1)}$$

- $0 \leq \mu < 1$ ($\mu = 0.9$) is the **momentum hyperparameter**

We now need to maintain the previous $\Delta \mathbf{w}^{(\tau-2)}$ update direction

Intuition: The solution **"picks up speed"** along the direction of repeated updates.

How Momentum Helps: Flat Directions

In **directions** that **are relatively flat** (e.g., is approx. constant ∇E):

$$\Delta \mathbf{w}^{(\infty)} = -\eta\, \nabla E + \mu \left(-\eta\, \nabla E + \mu \left(-\eta\, \nabla E + \mu\, (\dots)\right)\right)$$

$$\Delta \mathbf{w}^{(\infty)} = -\eta\, \nabla E \underbrace{(1 + \mu + \mu^2 + \mu^3 + \cdots)}_{\text{Geometric Series}} = -\left(\frac{\eta}{1-\mu}\right) \nabla E$$

The **momentum parameter μ increases** the effective learning rate

- Remember that $0 \leq \mu < 1$ and therefore $\eta \leq \left(\frac{\eta}{1-\mu}\right)$

How Momentum Helps: Oscillations

1846379

In **directions** that **are steep** (e.g., oscillating sign ∇E):

$$\Delta \mathbf{w}^{(\tau)} = -\eta\, \nabla E + \mu \left(-\eta\, (-\nabla E) + \mu \left(-\eta\, \nabla E + \mu\, (\ldots)\right)\right)$$

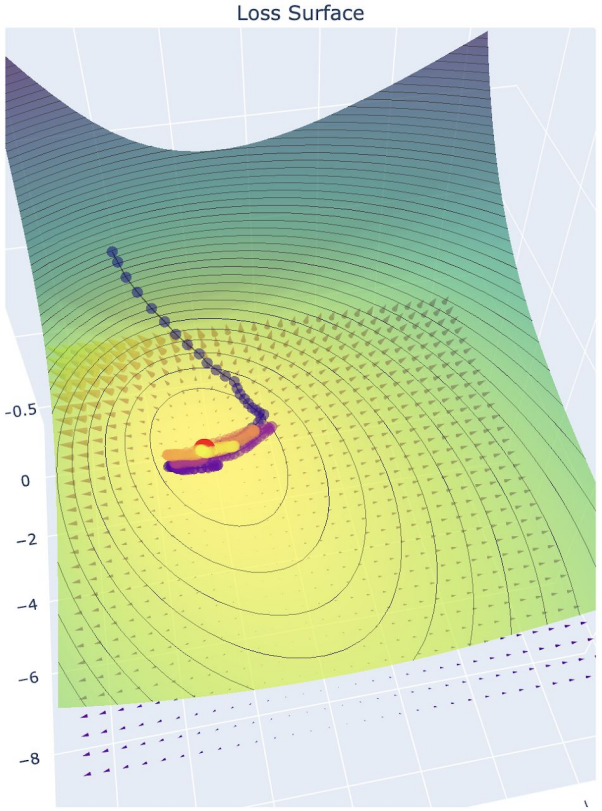
$$\Delta \mathbf{w}^{(\infty)} = -\eta\, \nabla E \underbrace{(1 - \mu + \mu^2 - \mu^3 + \cdots)}_{\text{Alternating Geometric Series}} = -\left(\frac{\eta}{1+\mu}\right) \nabla E$$

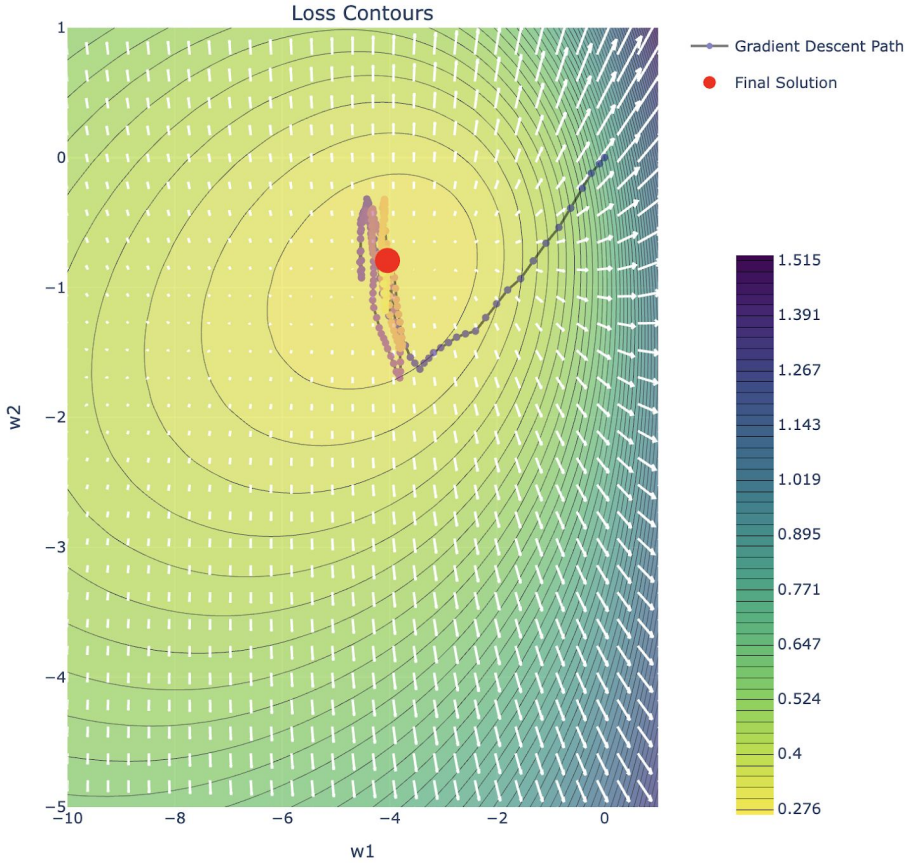
The **momentum parameter μ decrease** the effective learning rate

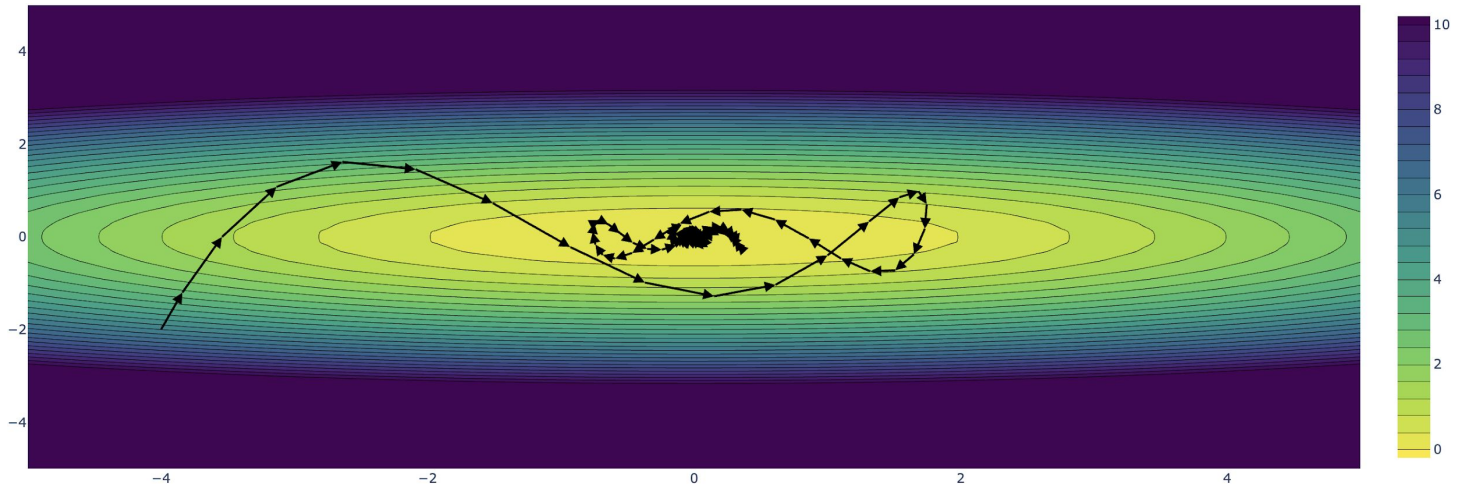
- Remember that $0 \leq \mu < 1$ and therefore $\left(\frac{\eta}{1+\mu}\right) \leq \eta$

Demo

Mini-batch Gradient Descent







Lecture 12

Optimization and Gradient Descent

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: Chapter 7